

## ♦ بخش ۱: مبانی جاوا

### ۱.۱ انواع داده

- 1 **byte**: نوع داده عددی ۸ بیتی با بازه -۱۲۸ تا ۱۲۷.
- 2 **short**: عدد صحیح ۱۶ بیتی.
- 3 **int**: عدد صحیح ۳۲ بیتی، پرکاربردترین نوع عددی.
- 4 **long**: عدد صحیح ۶۴ بیتی (با پسوند L).
- 5 **float**: عدد اعشاری ۳۲ بیتی (با پسوند f).
- 6 **double**: عدد اعشاری ۶۴ بیتی، پیش‌فرض اعداد اعشاری.
- 7 **char**: یک کاراکتر یونیکد ۱۶ بیتی.
- 8 **boolean**: مقدار **true** یا **false**.
- 9 **String**: کلاس برای نگهداری رشته متن، **immutable**.
- 10 **Array**: ساختار با اندازه ثابت برای نگهداری چند مقدار هم‌نوع.
- 11 **Enum**: نوع داده برای مجموعه‌ای ثابت از مقادیر نام‌دار.
- 12 **Boxing**: تبدیل **primitive** به **wrapper class**.
- 13 **Unboxing**: تبدیل **wrapper class** به **primitive**.
- 14 **Autoboxing**: boxing خودکار توسط کامپایلر.
- 15 **Wrapper Classes**: کلاس‌های پوششی مثل **Integer** برای **primitive**‌ها.
- 16 **Integer**: wrapper کلاس **int**.
- 17 **IEEE 754**: استاندارد نمایش اعداد اعشاری در حافظه.
- 18 **BigDecimal**: کلاس برای محاسبات اعشاری **دقیق** (بدون خطای شناوری).
- 19 **String Pool**: ناحیه‌ای در **heap** برای ذخیره **String**‌های **literal** و **reuse** آن‌ها.
- 20 **Immutable**: تغییرناپذیر؛ مثل **String**.
- 21 **StringBuilder**: ساخت رشته **mutable** و **سریع** (غیر **thread-safe**).
- 22 **StringBuffer**: ساخت رشته **mutable** و **thread-safe** (کندتر).
- 23 **Thread-Safe**: امن برای استفاده هم‌زمان چند **thread**.
- 24 **Mutable**: قابل تغییر پس از ساخت.
- 25 **Stack**: ناحیه حافظه برای متغیرهای محلی و فراخوانی متدها.

- 26 Heap: ناحیه حافظه برای آجکت‌ها.
- 27 NullPointerException: خطای دسترسی به reference با مقدار **null**.
- 28 Strongly Typed: زبان با قواعد سختگیرانه نوع داده.
- 29 Statically Typed: بررسی نوع در زمان کامپایل، نه اجرا.

## ۱.۲ عملگرها

- 1 <<: شیفت راست با حفظ علامت.
- 2 <<<: شیفت راست بدون حفظ علامت (پُر کردن با صفر).
- 3 &: AND بیتی / منطقی (هر دو طرف ارزیابی می‌شوند).
- 4 &&: AND منطقی با short-circuit.
- 5 |: OR بیتی / منطقی.
- 6 ||: OR منطقی با short-circuit.
- 7 Short-circuit Evaluation: ارزیابی شرط دوم فقط در صورت نیاز.

## ۱.۳ ساختارهای کنترلی

- 1 switch-case: انتخاب چندشاخه‌ای بر اساس مقدار.
- 2 Enhanced switch: نسخه جدید switch با arrow و بدون fall-through.
- 3 fall-through: اجرای ادامه case‌ها وقتی break نباشد.
- 4 break: خروج از حلقه یا switch.
- 5 default: شاخه پیش‌فرض switch.
- 6 for: حلقه با شمارنده.
- 7 for-each: پیمایش ساده عناصر آرایه/کالکشن.
- 8 while: حلقه با شرط قبل از اجرا.
- 9 do-while: حلقه با شرط بعد از اجرا (حداقل یک بار اجرا).
- 10 if / else: شرط ساده.

## ۱.۴ آرایه‌ها

- 1 Array: مجموعه با اندازه ثابت و هم‌نوع.

- 2 Fixed Size: اندازه آرایه پس از ساخت تغییر نمی‌کند.
- 3 **ArrayIndexOutOfBoundsException**: خطای دسترسی به ایندکس خارج از محدوده.
- 4 **Unchecked Exception**: خطای زمان اجرا که کامپایلر اجبار به **catch** نمی‌کند.
- 5 **int[]**: آرایه یک‌بعدی **int**.
- 6 **int[][]**: آرایه دوبعدی **int**.
- 7 **matrix**: آرایه دوبعدی.
- 8 **Arrays.sort()**: مرتب‌سازی آرایه.
- 9 **Arrays.stream()**: تبدیل آرایه به **Stream**.
- 10 **Arrays.asList()**: تبدیل آرایه به **List** با اندازه ثابت.

#### ۱.۵ آرگومان‌های خط فرمان

- 1 **String[] args**: آرایه آرگومان‌های ورودی برنامه.
- 2 **main method**: نقطه ورود اجرای برنامه.

### ♦ بخش ۲: ورودی/خروجی (I/O) در جاوا

#### ۲.۱ معماری I/O

- 1 **Byte-Oriented: I/O**: مبتنی بر بایت برای داده‌های باینری.
- 2 **Character-Oriented: I/O**: مبتنی بر کاراکتر برای متن.
- 3 **InputStream**: کلاس پایه برای خواندن بایت.
- 4 **OutputStream**: کلاس پایه برای نوشتن بایت.
- 5 **Reader**: کلاس پایه برای خواندن کاراکتر.
- 6 **Writer**: کلاس پایه برای نوشتن کاراکتر.
- 7 **Stream**: جریان داده از مبدأ به مقصد.
- 8 **Buffer**: حافظه موقت برای افزایش سرعت I/O.

#### ۲.۲ Byte Streams

- **1 FileInputStream**: خواندن بایت از فایل.
- **2 FileOutputStream**: نوشتن بایت در فایل.
- **3 ByteArrayInputStream**: خواندن از آرایه بایت در حافظه.
- **4 ByteArrayOutputStream**: نوشتن در آرایه بایت در حافظه.
- **5 DataInputStream**: خواندن داده‌های primitive به صورت قابل حمل.
- **6 DataOutputStream**: نوشتن داده‌های primitive به صورت قابل حمل.
- **7 BufferedInputStream**: بافر کردن خواندن بایت برای سرعت بیشتر.
- **8 BufferedOutputStream**: بافر کردن نوشتن بایت.
- **9 ObjectInputStream**: خواندن آبجکت سریالایز شده.
- **10 ObjectOutputStream**: نوشتن آبجکت به صورت سریالایز.
- **11 FilterInputStream**: کلاس پایه برای streamهای فیلترکننده ورودی.
- **12 FilterOutputStream**: کلاس پایه برای streamهای فیلترکننده خروجی.
- **13 buffer**: حافظه موقت داده.
- **14 read()**: خواندن یک بایت/کاراکتر.
- **15 write()**: نوشتن یک بایت/کاراکتر.
- **16 IOException**: خطای عمومی I/O.

### ۲.۳ Reader/Writer

- **1 FileReader**: خواندن کاراکتر از فایل.
- **2 FileWriter**: نوشتن کاراکتر در فایل.
- **3 StringReader**: خواندن از رشته.
- **4 StringWriter**: نوشتن در رشته.
- **5 InputStreamReader**: پل بین **byte stream** و **character stream**.
- **6 OutputStreamWriter**: پل بین **byte stream** و **character stream** خروجی.
- **7 BufferedReader**: بافر کردن خواندن کاراکتر + **readLine**.
- **8 BufferedWriter**: بافر کردن نوشتن کاراکتر + **newLine**.
- **9 PrintWriter**: نوشتن متن با متدهای **print/println**.
- **10 readLine()**: خواندن یک خط کامل.

- 11 newLine(): درج خط جدید.
- 12 flush(): تخلیه بافر و نوشتن فوری.
- 13 UTF-8: کدگذاری یونیکد پرکاربرد.
- 14 StandardCharsets: کلاس ثابت‌های Charset مثل UTF\_8.

## Java NIO ۲.۴

- 1 java.nio: پکیج I/O غیرمسدودکننده و مبتنی بر بافر.
- 2 Files: کلاس ابزاری برای کار با فایل‌ها.
- 3 Paths: کلاس ابزاری برای ساخت Path.
- 4 Path: نماینده مسیر فایل/دایرکتوری.
- 5 Files.readAllLines(): خواندن همه خطوط فایل در List.
- 6 Files.write(): نوشتن داده در فایل.
- 7 Files.copy(): کپی فایل.
- 8 Files.deleteIfExists(): حذف فایل اگر وجود داشته باشد.
- 9 Files.exists(): بررسی وجود فایل.
- 10 Files.readString(): خواندن کل محتوای فایل به صورت String.
- 11 Files.writeString(): نوشتن String در فایل.
- 12 Files.list(): لیست کردن محتوای دایرکتوری به صورت Stream.
- 13 Files.createDirectories(): ساخت دایرکتوری‌ها به صورت بازگشتی.
- 14 Files.size(): اندازه فایل.
- 15 Files.getLastModifiedTime(): زمان آخرین تغییر فایل.
- 16 Files.probeContentType(): تشخیص نوع MIME فایل.
- 17 StandardCopyOption: گزینه‌های کپی مثل REPLACE\_EXISTING.
- 18 Stream<Path>: جریان مسیرها برای پردازش دایرکتوری.
- 19 Blocking: I/O مسدودکننده که thread منتظر می‌ماند.
- 20 Non-blocking: I/O غیرمسدودکننده با Selector.
- 21 Selector: انتخابگر کانال‌های آماده در NIO.
- 22 Buffer-oriented: مدل NIO بر پایه بافر.

## 23• Stream-oriented: مدل I/O کلاسیک بر پایه stream.

## ۲.۵ Serialization

- 1 **Serializable**: اینترفیس نشانه‌گذاری برای قابلیت سریالایز شدن.
- 2 **serialVersionUID**: شناسه نسخه کلاس برای سازگاری سریالایز.
- 3 **transient**: فیلدی که سریالایز نمی‌شود.
- 4 **static**: فیلد **static** سریالایز نمی‌شود.
- 5 **ObjectOutputStream**: نوشتن آبجکت سریالایز شده.
- 6 **ObjectInputStream**: خواندن آبجکت سریالایز شده.
- 7 **writeObject()**: متد نوشتن آبجکت.
- 8 **readObject()**: متد خواندن آبجکت.
- 9 **defaultWriteObject()**: نوشتن پیش‌فرض فیلدهای **non-transient**.
- 10 **defaultReadObject()**: خواندن پیش‌فرض فیلدها.
- 11 **ClassNotFoundException**: خطای پیدا نشدن کلاس هنگام **deserialize**.
- 12 **Deserialize**: بازسازی آبجکت از بایت.
- 13 **Serialize**: تبدیل آبجکت به بایت.

## ۲.۶ Console

- 1 **Scanner**: کلاس ساده برای خواندن ورودی.
- 2 **stream : System.in**: ورودی استاندارد.
- 3 **nextLine()**: خواندن یک خط **String**.
- 4 **nextInt()**: خواندن **int**.
- 5 **nextDouble()**: خواندن **double**.
- 6 **BufferedReader**: خواندن سریع‌تر ورودی متنی.
- 7 **InputStreamReader**: پل بین **byte** و **character** برای ورودی.
- 8 **Integer.parseInt()**: تبدیل **String** به **int**.
- 9 **Console**: کلاس خواندن ورودی از کنسول واقعی.
- 10 **System.console()**: دریافت **Console** (ممکن است **null** باشد).

- 11 readLine(): خواندن خط از کنسول.
- 12 readPassword(): خواندن رمز بدون نمایش.

#### ۲.۷ سوالات مصاحبه‌ای I/O

- 1 Try-with-Resources: بستن خودکار منابع در try.
- 2 AutoCloseable: اینترفیس برای منابع قابل بستن خودکار.
- 3 Suppressed Exceptions: استثنای ثانویه که در try-with-resources سرکوب می‌شوند.
- 4 Memory Leak: نشت حافظه به‌خاطر بسته نشدن منابع.

### ♦ بخش ۳: برنامه‌نویسی شی‌گرا (OOP)

#### ۳.۱ مفاهیم پایه

- 1 Class: قالب/نقشه برای ساخت آبجکت.
- 2 Object: نمونه ساخته‌شده از کلاس.
- 3 Constructor: متد ویژه برای مقداردهی اولیه آبجکت.
- 4 Default Constructor: سازنده بدون آرگومان که کامپایلر می‌سازد.
- 5 this: ارجاع به آبجکت جاری.
- 6 new: عملگر ساخت آبجکت.
- 7 Heap: ناحیه حافظه برای آبجکت‌ها.
- 8 Stack: ناحیه حافظه برای متغیرهای محلی و فراخوانی متد.
- 9 Reference: اشاره‌گر به آبجکت در heap.
- 10 Instance: نمونه‌ای از کلاس.
- 11 Field: متغیر عضو کلاس.
- 12 Method: رفتار/تابع عضو کلاس.

#### ۳.۲ کپسوله‌سازی (Encapsulation)

- 1 Encapsulation: پنهان‌سازی داده‌ها و دسترسی از طریق متد.

- 2 Access Modifiers: سطوح دسترسی (private, default, protected, public).
- 3 private: فقط داخل همان کلاس.
- 4 default: فقط داخل همان پکیج.
- 5 protected: همان پکیج + زیرکلاس‌ها.
- 6 public: از همه جا قابل دسترسی.
- 7 Getter: متد خواندن فیلد.
- 8 Setter: متد تغییر فیلد.
- 9 POJO: کلاس ساده جاوا بدون وابستگی خاص.
- 10 Java Bean: POJO با constructor بدون آرگومان، getter/setter و Serializable.

### ۳.۳ وراثت (Inheritance)

- 1 Inheritance: ارث‌بری ویژگی‌ها از کلاس والد.
- 2 extends: کلمه کلیدی وراثت کلاس.
- 3 IS-A: رابطه وراثت (Car IS-A Vehicle).
- 4 Superclass: کلاس والد.
- 5 Subclass: کلاس فرزند.
- 6 Parent Class: کلاس والد.
- 7 Child Class: کلاس فرزند.
- 8 Code Reusability: استفاده مجدد از کد.
- 9 Tight Coupling: وابستگی زیاد بین کلاس‌ها.
- 10 Fragile Base Class: مشکل شکستن زیرکلاس‌ها با تغییر والد.
- 11 Association: رابطه عمومی بین دو کلاس.
- 12 Aggregation: رابطه HAS-A ضعیف (جزء مستقل).
- 13 Composition: رابطه HAS-A قوی (جزء وابسته به کل).
- 14 HAS-A: رابطه مالکیت.
- 15 Uses a: رابطه استفاده موقت.
- 16 Whole-Part: رابطه کل-جزء.
- 17 Lifecycle Dependency: وابستگی چرخه حیات جزء به کل.

18• **UML**: زبان مدل‌سازی نموداری.

19• **Diamond Problem**: ابهام وراثت چندگانه.

20• **Multiple Inheritance**: ارث‌بری از چند **کلاس** (در جاوا فقط با **interface**).

### ۳.۴ چندریختی (Polymorphism)

1• **Polymorphism**: چندشکلی؛ یک نام، رفتارهای متفاوت.

2• **Compile-Time Polymorphism**: چندریختی زمان **کامپایل** (overloading).

3• **Runtime Polymorphism**: چندریختی زمان **اجرا** (overriding).

4• **Method Overloading**: چند متد هم‌نام با امضای متفاوت.

5• **Method Overriding**: بازنویسی متد والد در فرزند.

6• **@Override**: انوتیشن بررسی صحت overriding.

7• **Dynamic Method Dispatch**: انتخاب متد در زمان اجرا.

8• **Signature**: نام + پارامترهای متد.

### ۳.۵ انتزاع (Abstraction)

1• **Abstraction**: پنهان‌سازی جزئیات پیاده‌سازی.

2• **Abstract Class**: کلاس ناقص که نمی‌توان از آن نمونه ساخت.

3• **Interface**: قرارداد رفتاری بدون پیاده‌سازی.

4• **abstract**: کلمه کلیدی برای کلاس/متد انتزاعی.

5• **implements**: کلمه کلیدی پیاده‌سازی **interface**.

6• **default method**: متد پیش‌فرض در **interface** (جاوا ۸+).

7• **static method**: متد **static** در **interface**.

8• **Functional Interface**: **interface** با یک متد **abstract**.

9• **@FunctionalInterface**: انوتیشن تضمین تک‌متدی بودن.

10• **Contract**: قرارداد رفتاری که **interface** تعریف می‌کند.

### ۳.۶ مفاهیم تکمیلی

1• **static**: متعلق به کلاس نه نمونه.

- 2• **final**: غیرقابل تغییر (متغیر/متد/کلاس).
- 3• **static field**: فیلد مشترک بین همه نمونه‌ها.
- 4• **static method**: متد قابل فراخوانی بدون نمونه.
- 5• **final variable**: ثابت.
- 6• **final method**: متد غیرقابل override.
- 7• **final class**: کلاس غیرقابل ارث‌بری.
- 8• **main method**: نقطه ورود برنامه.
- 9• **JVM**: ماشین مجازی اجرای بایت‌کد جاوا.

## ♦ بخش ۴: مباحث پیشرفته پایه

### ۴.۱ مدیریت استثنایا (Exception Handling)

- 1• **Throwable**: کلاس پایه همه خطاها و استثنایا.
- 2• **Error**: خطای جدی سیستمی که معمولاً قابل بازیابی نیست.
- 3• **Exception**: خطای قابل مدیریت در برنامه.
- 4• **Checked Exception**: استثنای اجبار به **catch/throws** در کامپایل.
- 5• **Unchecked Exception**: استثنای زمان اجرا بدون اجبار کامپایلر.
- 6• **RuntimeException**: کلاس پایه استثنایا **unchecked**.
- 7• **OutOfMemoryError**: خطای تمام شدن حافظه **heap**.
- 8• **StackOverflowError**: خطای پر شدن **stack** (مثلاً بازگشت بی‌پایان).
- 9• **NullPointerException**: خطای دسترسی به **null**.
- 10• **FileNotFoundException**: خطای پیدا نشدن فایل.
- 11• **SQLException**: خطای مربوط به دیتابیس.
- 12• **try**: بلوک کد مشکوک به خطا.
- 13• **catch**: بلوک مدیریت استثنا.
- 14• **finally**: بلوکی که همیشه اجرا می‌شود.
- 15• **throw**: پرتاب استثنا.

- 16 **throws**: اعلام استثنا در امضای متد.
- 17 **System.exit()**: خروج فوری از **JVM** (**finally** اجرا نمی‌شود).
- 18 **Try-with-Resources**: بستن خودکار منابع.
- 19 **AutoCloseable**: اینترفیس منابع قابل بستن خودکار.
- 20 **Suppressed Exceptions**: استثناهای ثانویه سرکوب‌شده.

#### ۴.۲ جنریک‌ها (Generics)

- 1 **Generics**: نوع‌دهی پارامتری برای **type safety**.
- 2 **Type Safety**: جلوگیری از خطای نوع در زمان کامپایل.
- 3 **Type Erasure**: حذف اطلاعات جنریک در زمان کامپایل.
- 4 **Wildcard**: علامت ؟ برای نوع نامشخص.
- 5 **wildcard ?**: upper-bounded **T**: **extends** (خواندن).
- 6 **wildcard ?**: lower-bounded **T**: **super** (نوشتن).
- 7 **Upper-Bounded Wildcard**: محدود به **T** و زیرکلاس‌هایش.
- 8 **Lower-Bounded Wildcard**: محدود به **T** و والد‌هایش.
- 9 **PECS**: Producer Extends, Consumer Super
- 10 **Producer Extends**: برای تولیدکننده از **extends**.
- 11 **Consumer Super**: برای مصرف‌کننده از **super**.
- 12 **ClassCastException**: خطای **cast** نامعتبر.
- 13 **instanceof**: بررسی نوع آبجکت.
- 14 **List<String>**: لیست نوع‌دار **String**.
- 15 **List<Integer>**: لیست نوع‌دار **Integer**.

#### ۴.۳ کالکشن‌ها (Collections Framework)

- 1 **Collection**: اینترفیس پایه مجموعه‌ها.
- 2 **List**: مجموعه مرتب با ایندکس و تکراری مجاز.
- 3 **Set**: مجموعه بدون تکرار.
- 4 **Queue**: صف برای پردازش ترتیبی.

- **Map 5**: ساختار کلید-مقدار.
- **ArrayList 6**: پیاده‌سازی List با آرایه پویا.
- **LinkedList 7**: پیاده‌سازی List/Deque با لیست پیوندی.
- **HashSet: Set 8**: بدون ترتیب با hash.
- **LinkedHashSet: Set 9**: با حفظ ترتیب درج.
- **TreeSet: Set 10**: مرتب بر اساس درخت سرخ-سیاه.
- **HashMap: Map 11**: بدون ترتیب با hash.
- **LinkedHashMap: Map 12**: با حفظ ترتیب درج.
- **TreeMap: Map 13**: مرتب بر اساس درخت سرخ-سیاه.
- **Hashtable: Map 14**: قدیمی و thread-safe.
- **ConcurrentHashMap: Map 15**: thread-safe با قفل سطح bucket.
- **Bucket 16**: سبد نگهداری عناصر در hash table.
- **Collision 17**: برخورد دو کلید در یک bucket.
- **Chaining 18**: حل collision با لیست پیوندی.
- **Red-Black Tree 19**: درخت متوازن خودتنظیم.
- **hashCode 20**: تولید کد هش آبجکت.
- **equals 21**: بررسی برابری منطقی.
- **Contract 22**: قرارداد hashCode/equals.
- **O(1) 23**: پیچیدگی ثابت.
- **O(n) 24**: پیچیدگی خطی.
- **O(log n) 25**: پیچیدگی لگاریتمی.
- **synchronized 26**: قفل همزمانی.
- **bucket-level locking 27**: قفل در سطح bucket در ConcurrentHashMap.
- **Node 28**: گره داخلی ساختار داده.
- **put 29**: افزودن به Map.
- **get 30**: خواندن از Map.
- **contains 31**: بررسی وجود عنصر.
- **add 32**: افزودن به کالکشن.

- 33• **remove()**: حذف از کالکشن.
- 34• **Comparator**: مقایسه‌گر خارجی.
- 35• **Comparable**: مقایسه‌گر داخلی (**compareTo**).
- 36• **Natural Ordering**: ترتیب طبیعی.
- 37• **Insertion Order**: ترتیب درج.
- 38• **Deque**: صف دوطرفه.
- 39• **ArrayDeque**: پیاده‌سازی Deque با آرایه.
- 40• **PriorityQueue**: صف اولویت‌دار.
- 41• **Stack**: پشته LIFO.
- 42• **push()**: افزودن به پشته.
- 43• **pop()**: حذف از پشته.
- 44• **peek()**: مشاهده بدون حذف.
- 45• **offer()**: افزودن به صف.
- 46• **poll()**: حذف از صف.
- 47• **enqueue()**: افزودن به صف.
- 48• **dequeue()**: حذف از صف.

## ♦ بخش ۵: جاوا پیشرفته (Advanced Java)

### ۵.۱ برنامه‌نویسی تابعی (Functional Programming)

- 1 • **Lambda Expression**: تابع بی‌نام و مختصر  $x \mapsto x + 1$ .
- 2 • **Anonymous Function**: تابع بدون نام.
- 3 • **Anonymous Inner Class**: کلاس داخلی بدون نام برای پیاده‌سازی سریع.
- 4 • **Runnable**: اینترفیس تابعی برای اجرای **thread**.
- 5 • **Functional Interface**: اینترفیس با یک متد **abstract**.
- 6 • **FunctionalInterface@**: انوتیشن تضمین تک‌متدی بودن.
- 7 • **java.util.function**: پکیج اینترفیس‌های تابعی.

- 8 Predicate<T>: تابع ورودی T و خروجی **boolean**.
- 9 Function<T, R>: تابع ورودی T و خروجی R.
- 10 Consumer<T>: تابع مصرف‌کننده بدون خروجی.
- 11 Supplier<T>: تابع تولیدکننده بدون ورودی.
- 12 Stream API: پردازش اعلانی مجموعه داده‌ها.
- 13 Stream: جریان داده برای عملیات زنجیره‌ای.
- 14 Intermediate Operation: عملیات میانی و lazy مثل filter/map.
- 15 Terminal Operation: عملیات پایانی که Stream را اجرا می‌کند.
- 16 Lazy Evaluation: اجرای عملیات فقط هنگام نیاز.
- 17 filter(): فیلتر عناصر.
- 18 map(): تبدیل هر عنصر.
- 19 sorted(): مرتب‌سازی.
- 20 distinct(): حذف تکراری‌ها.
- 21 limit(): محدود کردن تعداد.
- 22 collect(): جمع‌آوری نتیجه.
- 23 forEach(): اجرای عمل روی هر عنصر.
- 24 reduce(): کاهش به یک مقدار.
- 25 Collectors.toList(): جمع‌آوری در List.
- 26 Declarative: سبک اعلانی (چه چیزی، نه چگونه).
- 27 BinaryOperator: تابع دو ورودی هم‌نوع با خروجی هم‌نوع.
- 28 Memoization: کش کردن نتیجه محاسبات تکراری.

## ۵.۲ هم‌روندی (Concurrency)

- 1 Thread: ریسه اجرای سبک.
- 2 Runnable: اینترفیس بدون خروجی برای thread.
- 3 Callable<V>: اینترفیس با خروجی و امکان پرتاب استثنا.
- 4 run(): بدنه اجرای Runnable.
- 5 start(): شروع thread جدید.

- 6 **call**(): بدنه اجرای Callable.
- 7 **Race Condition**: شرایط رقابتی روی منابع مشترک.
- 8 **Shared Resource**: منبع مشترک بین threadها.
- 9 **synchronized**: قفل انحصاری برای همزمانی.
- 10 **Lock**: اینترفیس قفل پیشرفته.
- 11 **Deadlock**: بن بست بین threadها.
- 12 **tryLock**(): تلاش غیرمسدودکننده برای گرفتن قفل.
- 13 **Thread Pool**: مجموعه threadهای آماده استفاده مجدد.
- 14 **ExecutorService**: مدیریت thread pool.
- 15 **Executors.newFixedThreadPool**(): ساخت pool با اندازه ثابت.
- 16 **submit**(): ارسال task به executor.
- 17 **shutdown**(): خاموشی تدریجی executor.
- 18 **CompletableFuture**: API برنامه نویسی async در جاوا 8+.
- 19 **supplyAsync**(): اجرای async با خروجی.
- 20 **thenApply**(): تبدیل نتیجه async.
- 21 **thenAccept**(): مصرف نتیجه async.
- 22 **thenCombine**(): ترکیب دو CompletableFuture.
- 23 **Asynchronous**: ناهمزمان.
- 24 **Optional**: ظرف امن برای مقدار ممکن **null**.
- 25 **Optional.ofNullable**(): ساخت Optional از مقدار ممکن **null**.
- 26 **ifPresent**(): اجرا در صورت وجود مقدار.
- 27 **orElse**(): مقدار پیش فرض در صورت خالی بودن.
- 28 **orElseThrow**(): پرتاب استثنا در صورت خالی بودن.
- 29 **map**(): تبدیل مقدار داخل Optional.
- 30 **NullPointerException**: خطای **null**.
- 31 **Default Method**: متد پیش فرض **interface**.
- 32 **Static Method**: متد **static interface**.
- 33 **Interface**: قرارداد رفتاری.

- 34 **Vehicle**: مثال کلاس انتزاعی.
- 35 **Car**: مثال کلاس فرزند.
- 36 **start()**: متد شروع.
- 37 **stop()**: متد توقف.
- 38 **service()**: متد سرویس در Servlet.

## ♦ بخش ۶: ساختمان داده و الگوریتم

### ۶.۱ تحلیل پیچیدگی (Big O)

- 1 **Big O Notation**: نماد تحلیل رشد زمانی/فضایی الگوریتم.
- 2 **O(1)**: زمان ثابت.
- 3 **O(log n)**: زمان لگاریتمی.
- 4 **O(n)**: زمان خطی.
- 5 **O(n log n)**: زمان خطی-لگاریتمی.
- 6 **O(n<sup>2</sup>)**: زمان درجه دوم.
- 7 **O(2<sup>n</sup>)**: زمان نمایی.
- 8 **Constant**: پیچیدگی ثابت.
- 9 **Logarithmic**: پیچیدگی لگاریتمی.
- 10 **Linear**: پیچیدگی خطی.
- 11 **Linearithmic**: پیچیدگی  $n \log n$ .
- 12 **Quadratic**: پیچیدگی درجه دوم.
- 13 **Exponential**: پیچیدگی نمایی.
- 14 **Scalability**: مقیاس پذیری.
- 15 **Time Complexity**: پیچیدگی زمانی.
- 16 **Space Complexity**: پیچیدگی فضایی.
- 17 **Amortized**: تحلیل میانگین هزینه در طول زمان.
- 18 **binarySearch**: جستجوی دودویی.

- 19• QuickSort: مرتب‌سازی سریع.
- 20• MergeSort: مرتب‌سازی ادغامی.
- 21• BubbleSort: مرتب‌سازی حبابی.

## ۶.۲ ساختمان داده‌ها

- 1• Array: آرایه با دسترسی  $O(1)$ .
- 2• Linked List: لیست پیوندی با درج/حذف سریع.
- 3• Stack: پشته LIFO.
- 4• Queue: صف FIFO.
- 5• Hash Map: نگاشت کلید-مقدار با جستجوی  $O(1)$ .
- 6• Hash Table: جدول هش.
- 7• LIFO: آخرین ورود، اولین خروج.
- 8• FIFO: اولین ورود، اولین خروج.
- 9• Push: افزودن به پشته.
- 10• Pop: حذف از پشته.
- 11• Peek: مشاهده بدون حذف.
- 12• Enqueue: افزودن به صف.
- 13• Dequeue: حذف از صف.
- 14• Bucket: سبد در hash table.
- 15• Collision: برخورد کلیدها.
- 16• Chaining: حل برخورد با زنجیره.
- 17• Red-Black Tree: درخت متوازن خودتنظیم.

## ۶.۳ الگوریتم‌ها

- 1• Linear Search: جستجوی خطی  $O(n)$ .
- 2• Binary Search: جستجوی دودویی  $O(\log n)$  روی داده مرتب.
- 3• Bubble Sort: مرتب‌سازی حبابی  $O(n^2)$ .
- 4• Quick Sort: مرتب‌سازی سریع با pivot.

- 5 Merge Sort: مرتب‌سازی ادغامی پایدار.
- 6 Divide and Conquer: تقسیم و غلبه.
- 7 Dynamic Programming: برنامه‌نویسی پویا برای مسائل بهینه‌سازی.
- 8 Memoization: کش نتایج برای جلوگیری از محاسبه تکراری.
- 9 Greedy: الگوریتم حریصانه.
- 10 Backtracking: عقب‌گرد برای جستجوی کامل.
- 11 Dijkstra: کوتاه‌ترین مسیر در گراف وزن‌دار.
- 12 Eight Queens: مسئله هشت وزیر.
- 13 Sudoku: حل سودوکو.
- 14 Fibonacci: دنباله فیبوناچی.
- 15 Pivot: عنصر محوری در QuickSort.
- 16 Stable Sort: مرتب‌سازی پایدار (حفظ ترتیب عناصر برابر).

## ♦ بخش ۷: شبکه‌های کامپیوتری (Computer Networks)

### ۷.۱ مدل OSI و TCP/IP

- 1 OSI Model: مدل هفت‌لایه مرجع شبکه.
- 2 TCP/IP Model: مدل چهارلایه عملی اینترنت.
- 3 Application Layer: لایه برنامه‌های کاربر (HTTP, FTP, DNS).
- 4 Presentation Layer: لایه قالب‌بندی و رمزنگاری داده.
- 5 Session Layer: لایه مدیریت نشست.
- 6 Transport Layer: لایه انتقال (TCP/UDP) end-to-end.
- 7 Network Layer: لایه مسیریابی (IP).
- 8 Data Link Layer: لایه انتقال فریم (MAC, Ethernet).
- 9 Physical Layer: لایه سیگنال‌های فیزیکی.
- 10 HTTP: پروتکل انتقال ابرمتن.
- 11 HTTPS: HTTP امن با TLS.

- 12• **FTP**: پروتکل انتقال فایل.
- 13• **SMTP**: پروتکل ارسال ایمیل.
- 14• **DNS**: سیستم نام دامنه.
- 15• **SSL/TLS**: پروتکل‌های رمزنگاری ارتباط.
- 16• **ASCII**: کدگذاری کاراکتر متنی.
- 17• **JPEG**: فرمت فشرده تصویر.
- 18• **NetBIOS: API**: ارتباطی قدیمی شبکه‌های میکروسافتی.
- 19• **RPC**: فراخوانی رویه از راه دور.
- 20• **TCP**: پروتکل انتقال مطمئن و اتصال‌گرا.
- 21• **UDP**: پروتکل انتقال سریع و بدون اتصال.
- 22• **IP**: پروتکل آدرس‌دهی و مسیریابی.
- 23• **ICMP**: پروتکل پیام‌های **کنترلی** (ping).
- 24• **Router**: مسیریاب بین شبکه‌ها.
- 25• **Ethernet**: فناوری شبکه محلی سیمی.
- 26• **Switch**: سوئیچ لایه دو.
- 27• **MAC**: آدرس فیزیکی کارت شبکه.
- 28• **Hub**: هاب ساده لایه یک.

## ۷.۲ پروتکل‌های TCP و UDP

- 1 • **TCP**: اتصال‌گرا، مطمئن، با ترتیب.
- 2 • **UDP**: بدون اتصال، سریع، بدون تضمین.
- 3 • **Connection-Oriented**: نیازمند برقراری اتصال.
- 4 • **Connectionless**: بدون برقراری اتصال.
- 5 • **Acknowledgement**: تأیید دریافت داده.
- 6 • **Flow Control**: کنترل جریان داده.
- 7 • **Congestion Control**: کنترل ازدحام شبکه.
- 8 • **Overhead**: سرباره اضافی پروتکل.
- 9 • **Three-way Handshake**: برقراری اتصال TCP در سه مرحله.

• 10 SYN: بسته درخواست اتصال.

• 11 SYN-ACK: پاسخ تأیید اتصال.

• 12 ACK: تأیید دریافت.

### HTTP ۷.۳

• 1 HTTP: پروتکل درخواست-پاسخ بدون حالت.

• 2 HTTPS: HTTP روی TLS.

• 3 GET: دریافت منبع.

• 4 POST: ایجاد منبع.

• 5 PUT: جایگزینی کامل منبع.

• 6 PATCH: به‌روزرسانی جزئی منبع.

• 7 DELETE: حذف منبع.

• 8 HEAD: دریافت فقط هدر.

• 9 OPTIONS: دریافت متدهای مجاز.

• 10 Idempotent: متدی که تکرارش نتیجه یکسان دارد.

• 11 Safe: متدی بدون تغییر سرور.

• 12 Status Code: کد وضعیت پاسخ.

• 13 OK 200: موفقیت.

• 14 Created 201: منبع ساخته شد.

• 15 No Content 204: موفقیت بدون بدنه.

• 16 Moved Permanently 301: انتقال دائمی.

• 17 Found 302: انتقال موقت.

• 18 Bad Request 400: درخواست نامعتبر.

• 19 Unauthorized 401: نیاز به احراز هویت.

• 20 Forbidden 403: دسترسی ممنوع.

• 21 Not Found 404: منبع پیدا نشد.

• 22 Method Not Allowed 405: متد غیرمجاز.

• 23 Internal Server Error 500: خطای داخلی سرور.

- 24 502 Bad Gateway: پاسخ نامعتبر از سرور بالادست.
- 25 503 Service Unavailable: سرویس در دسترس نیست.
- 26 Header: بخش متادیتای درخواست/پاسخ.
- 27 Body: بخش داده اصلی.
- 28 Content-Type: نوع محتوای بدنه.
- 29 Authorization: هدر احراز هویت.
- 30 Bearer Token: توکن احراز هویت.
- 31 Accept: نوع محتوای قابل قبول.
- 32 Host: نام میزبان مقصد.

#### ۷.۴ مفاهیم پیشرفته شبکه

- 1 REST API: مبتنی بر منابع و متدهای HTTP.
- 2 Stateless: بدون نگهداری وضعیت بین درخواست‌ها.
- 3 Resource-Based: مبتنی بر منابع.
- 4 HTTP Verbs: متدهای HTTP.
- 5 JSON: فرمت داده سبک.
- 6 XML: فرمت داده نشانه‌گذاری.
- 7 HTTP/1.1: نسخه کلاسیک HTTP با keep-alive.
- 8 HTTP/2: نسخه چندگانه با multiplexing.
- 9 Head-of-Line Blocking: مسدود شدن صف به‌خاطر یک درخواست.
- 10 Multiplexing: ارسال چند درخواست روی یک اتصال.
- 11 Header Compression: فشرده‌سازی هدرها.
- 12 HPACK: الگوریتم فشرده‌سازی هدر در HTTP/2.
- 13 Server Push: ارسال پیش‌دستانه منابع توسط سرور.
- 14 Binary: پروتکل دودویی (HTTP/2).
- 15 Text-based: پروتکل متنی (HTTP/1.1).
- 16 CORS: اشتراک منابع بین دامنه‌ای.
- 17 Same-Origin Policy: سیاست امنیتی مرورگر.

- 18• **Preflight**: درخواست OPTIONS پیش از درخواست اصلی.
- 19• **Access-Control-Allow-Origin**: هدر مجوز CORS.
- 20• **SSL/TLS**: لایه امنیت ارتباط.
- 21• **Certificate**: گواهی دیجیتال سرور.
- 22• **DNS**: سیستم نام دامنه.
- 23• **IP Address**: آدرس عددی دستگاه.
- 24• **Domain Name**: نام خوانا مثل **google.com**.
- 25• **DNS Resolver**: حل کننده نام دامنه.
- 26• **DNS Lookup**: فرآیند تبدیل نام به IP.
- 27• **TCP Connection**: اتصال TCP.
- 28• **TLS Handshake**: دست دادن برای برقراری امنیت.

## ♦ بخش ۸: سیستم عامل (Operating System)

### ۸.۱ پردازش و ریس

- 1• **Process**: برنامه در حال اجرا با فضای حافظه مستقل.
- 2• **Thread**: ریس سبک داخل process با حافظه مشترک.
- 3• **Heap**: ناحیه حافظه پویا برای آبجکت ها.
- 4• **Stack**: ناحیه حافظه برای متغیرهای محلی و فراخوانی ها.
- 5• **Code Segment**: بخش کد برنامه در حافظه.
- 6• **Data Segment**: بخش داده های سراسری و **static**.
- 7• **IPC (Inter-Process Communication)**: ارتباط بین process ها.
- 8• **Socket**: نقطه پایانی ارتباط شبکه ای/بین پردازش های.
- 9• **Pipe**: کانال ارتباطی یک طرفه بین process ها.
- 10• **Race Condition**: شرایط رقابتی روی منابع مشترک.
- 11• **Lightweight Process**: نام دیگر thread.
- 12• **Context Switch**: تعویض context بین process/thread ها.

13 • Page Tables: جدول نگاشت صفحات حافظه مجازی.

14 • TLB: کش ترجمه آدرس‌های مجازی به فیزیکی.

15 • Program Counter: اشاره‌گر به دستور بعدی.

16 • Stack Pointer: اشاره‌گر به بالای stack.

## ۸.۲ مدیریت حافظه

1 • Virtual Memory: حافظه مجازی بزرگ‌تر از RAM فیزیکی.

2 • RAM: حافظه اصلی فرار.

3 • Disk: حافظه جانبی غیرفرار.

4 • Memory Layout: چیدمان حافظه برنامه.

5 • Text Segment: بخش کد اجرایی.

6 • Data Segment: بخش داده‌های سراسری.

7 • Stack: ناحیه متغیرهای محلی.

8 • Heap: ناحیه تخصیص پویا.

9 • Dynamic Memory Allocation: تخصیص حافظه در زمان اجرا.

10 • Garbage Collector: جمع‌آورنده خودکار حافظه بی‌استفاده.

11 • StackOverflowError: خطای پر شدن stack.

## ۸.۳ هم‌روندی و بن‌بست

1 • Deadlock: بن‌بست؛ انتظار چرخشی threadها.

2 • Coffman Conditions: چهار شرط لازم برای deadlock.

3 • Mutual Exclusion: انحصار منابع.

4 • Hold and Wait: نگه‌داشتن منبع و انتظار برای دیگری.

5 • No Preemption: عدم امکان گرفتن منبع از thread.

6 • Circular Wait: انتظار چرخشی.

7 • tryLock(): تلاش غیرمسدودکننده برای گرفتن قفل.

8 • ReentrantLock: قفل قابل ورود مجدد.

9 • Semaphore: شمارنده کنترل دسترسی به منابع.

10• Lock: قفل هم‌زمانی.

#### ۸.۴ زمان‌بندی CPU

- 1• CPU Scheduling: انتخاب process بعدی برای اجرا.
- 2• Context Switch: تعویض process/thread روی CPU.
- 3• Dispatcher: مازول تعویض context.
- 4• Process Context Switch: تعویض کامل process.
- 5• Thread Context Switch: تعویض سبک thread.
- 6• TLB Flush: پاک کردن TLB هنگام تعویض.

#### ۸.۵ مفاهیم سیستمی

- 1• Virtual Memory: حافظه مجازی.
- 2• Cache: حافظه موقت سریع.
- 3• Redis: دیتابیس/کش درون حافظه‌ای.
- 4• Ehcache: کتابخانه کش جاوا.
- 5• Lock: قفل.
- 6• Semaphore: سمافور.

### ♦ بخش ۹: امنیت (Security)

#### ۹.۱ مفاهیم پایه

- 1• Authentication: احراز هویت (چه کسی هستی؟).
- 2• Authorization: مجوزدهی (چه کاری می‌توانی بکنی؟).
- 3• Unauthorized 401: عدم احراز هویت.
- 4• Forbidden 403: احراز هویت شده اما بدون مجوز.
- 5• Plain Text: متن بدون رمزنگاری.
- 6• Hashing: تبدیل یک‌طرفه داده به هش.

- **7 Salt**: مقدار تصادفی افزوده به رمز قبل از hash.
- **8 Rainbow Table Attack**: حمله با جدول هش‌های آماده.
- **9 BCrypt**: الگوریتم هش رمز کند و امن.
- **10 SCrypt**: الگوریتم هش مقاوم به حافظه.
- **11 Argon2**: برنده مسابقه هش رمز، مقاوم به GPU.
- **12 BCryptPasswordEncoder**: پیاده‌سازی BCrypt در Spring Security.
- **13 Brute-Force**: حمله آزمون همه ترکیب‌ها.
- **14 One-way**: تابع غیرقابل برگشت.

## JWT ۹.۲

- **1 JWT (JSON Web Token)**: توکن استاندارد برای احراز هویت بدون حالت.
- **2 RFC 7519**: استاندارد رسمی JWT.
- **3 Claims**: ادعاهای داخل payload.
- **4 Stateless**: بدون نگهداری session در سرور.
- **5 Session**: نگهداری وضعیت کاربر در سرور.
- **6 Header**: بخش الگوریتم و نوع توکن.
- **7 Payload**: بخش داده‌های JWT.
- **8 Signature**: امضای دیجیتال JWT.
- **9 Base64-encoded**: کدگذاری Base64 بخش‌های JWT.
- **10 HMACSHA256**: الگوریتم امضای متقارن.
- **11 SECRET\_KEY**: کلید مخفی امضا.
- **12 Authorization: Bearer**: هدر ارسال توکن.
- **13 Tamper**: دستکاری توکن.
- **14 Local Storage**: ذخیره‌سازی سمت مرورگر.
- **15 Cookie**: ذخیره‌سازی کوچک سمت کلاینت.
- **16 Expiration**: زمان انقضای توکن.
- **17 JwtService**: سرویس ساخت/اعتبارسنجی JWT.
- **18 Jwts.builder()**: سازنده JWT در JJWT.

- 19 • **Jwts.parser()**: تجزیه‌کننده JWT.
- 20 • **JwtException**: خطای مربوط به JWT.
- 21 • **JwtAuthenticationFilter**: فیلتر احراز هویت با JWT.
- 22 • **OncePerRequestFilter**: فیلتر اجراشده یک بار در هر درخواست.
- 23 • **SecurityContextHolder**: نگهدارنده SecurityContext.
- 24 • **UsernamePasswordAuthenticationToken**: توکن احراز هویت Spring.

### ۹.۳ آسیب‌پذیری‌ها (OWASP)

- 1 • **SQL Injection**: تزریق SQL از ورودی کاربر.
- 2 • **PreparedStatement**: جلوگیری از SQL Injection با پارامتر.
- 3 • **Parameterized Queries**: کوئری پارامتری.
- 4 • **XSS (Cross-Site Scripting)**: تزریق اسکریپت در مرورگر.
- 5 • **CSRF (Cross-Site Request Forgery)**: جعل درخواست از سایت دیگر.
- 6 • **CSRF Token**: توکن ضد CSRF.
- 7 • **Escape**: خنثی‌سازی کاراکترهای خاص.
- 8 • **Encode**: کدگذاری خروجی.
- 9 • **Thymeleaf**: قالب‌ساز امن Spring.
- 10 • **OWASP Top 10**: فهرست ده آسیب‌پذیری مهم وب.

### ۹.۴ Spring Security

- 1 • **Spring Security**: فریم‌ورک امنیت Spring.
- 2 • **UsernamePasswordAuthenticationFilter**: فیلتر ورود با نام کاربری/رمز.
- 3 • **AuthenticationManager**: مدیر احراز هویت.
- 4 • **PasswordEncoder**: اینترفیس هش رمز.
- 5 • **BCryptPasswordEncoder**: پیاده‌سازی BCrypt.
- 6 • **SCryptPasswordEncoder**: پیاده‌سازی SCrypt.
- 7 • **Argon2PasswordEncoder**: پیاده‌سازی Argon2.
- 8 • **UserDetailsService**: سرویس بارگذاری کاربر.

- 9 **UserDetails**: اینترفیس اطلاعات کاربر.
- 10 **UsernameNotFoundException**: خطای پیدا نشدن کاربر.
- 11 **SimpleGrantedAuthority**: پیاده‌سازی ساده **Authority**.
- 12 **SecurityContext**: نگهدارنده اطلاعات احراز هویت.
- 13 **SecurityContextHolder**: دسترسی به **SecurityContext**.
- 14 **JwtService**: سرویس JWT.
- 15 **JwtAuthenticationFilter**: فیلتر JWT.
- 16 **SecurityFilterChain**: زنجیره فیلترهای امنیتی.
- 17 **EnableWebSecurity@**: فعال‌سازی امنیت وب.
- 18 **EnableMethodSecurity@**: فعال‌سازی امنیت سطح متد.
- 19 **authorizeHttpRequests**: پیکربندی مجوز درخواست‌ها.
- 20 **requestMatchers**: تطبیق مسیرها.
- 21 **permitAll**: اجازه به همه.
- 22 **hasRole**: بررسی نقش.
- 23 **hasAnyRole**: بررسی چند نقش.
- 24 **authenticated**: فقط کاربران احراز هویت شده.
- 25 **csrf**: پیکربندی CSRF.
- 26 **sessionManagement**: مدیریت **session**.
- 27 **SessionCreationPolicy.STATELESS**: بدون ساخت **session**.
- 28 **addFilterBefore**: افزودن فیلتر قبل از فیلتر دیگر.
- 29 **PreAuthorize@**: بررسی مجوز قبل از اجرای متد.
- 30 **Secured@**: بررسی نقش ساده.
- 31 **hasRole('ADMIN')**: نیاز به نقش ADMIN.
- 32 **hasAnyRole('USER', 'ADMIN')**: نیاز به یکی از نقش‌ها.
- 33 **authentication.principal.id**: دسترسی به id کاربر جاری.
- 34 **AuthenticationEntryPoint**: نقطه ورود برای احراز هویت.
- 35 **commence()**: متد شروع چالش احراز هویت.
- 36 **SC\_UNAUTHORIZED**: کد وضعیت 401.

37. **APPLICATION\_JSON\_VALUE**: MediaType: نوع محتوای JSON.

## ♦ بخش ۱۰: دیزاین پترن‌ها (Design Patterns)

### ۱۰.۱ Creational (ایجاد)

- **1 Singleton**: تنها یک نمونه از کلاس در کل برنامه.
- **2 Factory Method**: ساخت آبجکت با متد کارخانه‌ای و بدون افشای منطق ساخت.
- **3 Abstract Factory**: ساخت خانواده‌ای از آبجکت‌های مرتبط.
- **4 Builder**: ساخت گام‌به‌گام آبجکت پیچیده.
- **5 Prototype**: ساخت آبجکت جدید با clone از نمونه موجود.
- **6 GameManager**: مثال Singleton برای مدیریت بازی.
- **7 ConfigManager**: مثال Singleton برای تنظیمات.
- **8 InputManager**: مثال Singleton برای ورودی.
- **9 GameConfig**: مثال Singleton برای پیکربندی بازی.
- **10 Logger**: مثال Singleton برای لاگ.
- **11 EnemyFactory**: مثال Factory برای ساخت دشمن.
- **12 Orc**: مثال محصول Factory.
- **13 Troll**: مثال محصول Factory.
- **14 Transport**: مثال Abstract Factory برای حمل‌ونقل.
- **15 Truck**: مثال محصول Abstract Factory.
- **16 Ship**: مثال محصول Abstract Factory.
- **17 Airplane**: مثال محصول Abstract Factory.
- **18 Logistics**: مثال Abstract Factory.
- **19 RoadLogistics**: کارخانه جاده‌ای.
- **20 SeaLogistics**: کارخانه دریایی.
- **21 SkyLogistics**: کارخانه هوایی.
- **22 GUIFactory**: کارخانه Abstract برای UI.
- **23 Button**: مثال محصول UI.

- 24• **Checkbox**: مثال محصول UI.
- 25• **WindowsButton**: دکمه ویندوزی.
- 26• **MacButton**: دکمه مک.
- 27• **WindowsFactory**: کارخانه UI ویندوز.
- 28• **MacFactory**: کارخانه UI مک.
- 29• **Character**: مثال Builder برای شخصیت بازی.
- 30• **Builder**: سازنده گام به گام.
- 31• **House**: مثال Builder برای خانه.
- 32• **HousePlan**: نقشه خانه.
- 33• **HouseBuilder**: سازنده خانه.
- 34• **IglooHouseBuilder**: سازنده ایگلو.
- 35• **TipiHouseBuilder**: سازنده تیپی.
- 36• **CivilEngineer**: مهندس ناظر ساخت.
- 37• **clone()**: متد کلون کردن آبجکت.
- 38• **Cloneable**: اینترفیس نشانه گذاری برای clone.
- 39• **CloneNotSupportedException**: خطای عدم پشتیبانی clone.

## ۱۰.۲ Structural (ساختاری)

- 1 • **Adapter**: تبدیل **interface** یک کلاس به **interface** مورد انتظار.
- 2 • **Bridge**: جدا کردن abstraction از implementation.
- 3 • **Composite**: ساختار درختی از آبجکت ها.
- 4 • **Decorator**: افزودن رفتار به آبجکت به صورت پویا.
- 5 • **Facade: interface** ساده برای زیرسیستم پیچیده.
- 6 • **Flyweight**: اشتراک گذاری آبجکت های مشابه برای صرفه جویی حافظه.
- 7 • **Proxy**: نماینده/واسط برای کنترل دسترسی.
- 8 • **GameInput**: مثال **interface** ورودی بازی.
- 9 • **ExternalInputSystem**: سیستم ورودی خارجی.
- 10• **InputAdapter**: آداپتور ورودی.

- 11• **Renderer**: مثال Bridge برای رندر.
- 12• **VectorRenderer**: رندر برداری.
- 13• **RasterRenderer**: رندر رستری.
- 14• **Circle**: مثال Composite/Bridge.
- 15• **GameObject**: مثال Composite.
- 16• **Sprite**: مثال Composite.
- 17• **Group**: گروه آبجکت‌ها در Composite.
- 18• **Weapon**: مثال Decorator.
- 19• **BasicSword**: شمشیر پایه.
- 20• **FireDecorator**: دکوراتور آتش.
- 21• **AudioSystem**: مثال Facade.
- 22• **PhysicsSystem**: مثال Facade.
- 23• **GameEngineFacade**: فاساد موتور بازی.
- 24• **TreeType**: مثال Flyweight.
- 25• **Tree**: مثال Flyweight.
- 26• **Image**: مثال Proxy.
- 27• **ReallImage**: تصویر واقعی.
- 28• **ImageProxy**: پروکسی تصویر.

### ۱۰.۳ Behavioral (رفتاری)

- 1 • **Strategy**: انتخاب الگوریتم در زمان اجرا.
- 2 • **Observer**: اطلاع‌رسانی تغییرات به observerها.
- 3 • **Command**: کپسوله‌سازی درخواست به صورت آبجکت.
- 4 • **Iterator**: پیمایش عناصر بدون افشای ساختار.
- 5 • **Mediator**: واسطه ارتباط بین آبجکت‌ها.
- 6 • **Memento**: ذخیره و بازیابی وضعیت.
- 7 • **State**: تغییر رفتار بر اساس وضعیت.
- 8 • **Template Method**: اسکلت الگوریتم با گام‌های قابل بازنویسی.

- 9 Visitor: عملیات جدید بدون تغییر کلاس‌ها.
- 10 Chain of Responsibility: زنجیره handlerها.
- 11 PaymentStrategy: مثال Strategy پرداخت.
- 12 CreditCardPayment: پرداخت با کارت.
- 13 CryptoPayment: پرداخت با کریپتو.
- 14 PaymentProcessor: پردازشگر پرداخت.
- 15 RouteStrategy: استراتژی مسیریابی.
- 16 Navigator: ناوبر.
- 17 CarRouteStrategy: مسیر ماشین.
- 18 WalkingRouteStrategy: مسیر پیاده.
- 19 Observer: ناظر.
- 20 Player: مثال Observer.
- 21 Command: فرمان.
- 22 JumpCommand: فرمان پرش.
- 23 InputHandler: مدیریت ورودی.
- 24 Television: مثال Command.
- 25 TurnOnCommand: فرمان روشن.
- 26 TurnOffCommand: فرمان خاموش.
- 27 RemoteControllerInvoker: کنترل از راه دور.
- 28 Iterator: پیمایشگر.
- 29 Mediator: واسطه.
- 30 Component: جزء در Mediator.
- 31 GameState: وضعیت بازی در State.
- 32 Game: بازی.
- 33 PlayerState: وضعیت بازیکن.
- 34 StandingState: ایستاده.
- 35 RunningState: در حال دویدن.
- 36 DocumentState: وضعیت سند.

- 37 Document: سند.
- 38 DraftState: پیش نویس.
- 39 ModerationState: در حال بررسی.
- 40 PublishState: منتشرشده.
- 41 GameLoop: حلقه بازی.
- 42 Visitor: بازدیدکننده.
- 43 Element: عنصر در Visitor.
- 44 Handler: هندلر در Chain of Responsibility.
- 45 AuthHandler: هندلر احراز هویت.

#### ۱۰.۴ Concurrency (همروندی)

- 1 Active Object: جدا کردن فراخوانی متد از اجرا با صف پیام.
- 2 Monitor Object: قفل و همزمانی داخلی در آبجکت.

## ♦ بخش ۱۱: Spring Boot

### ۱۱.۱ چرخه حیات Bean

- 1 Bean: آبجکت مدیریت شده توسط Spring Container.
- 2 Instantiation: ساخت نمونه Bean.
- 3 Dependency Injection: تزریق وابستگی ها به Bean.
- 4 Initialization: مرحله مقداردهی اولیه Bean.
- 5 @PostConstruct: انوتیشن متد اجراشده بعد از مقداردهی اولیه.
- 6 afterPropertiesSet: متد InitializingBean اجراشده بعد از پر شدن property ها.
- 7 @PreDestroy: انوتیشن متد اجراشده قبل از نابودی Bean.
- 8 DisposableBean: اینترفیس با متد destroy.
- 9 InitializingBean: اینترفیس با متد afterPropertiesSet.
- 10 BeanNameAware: اینترفیس دریافت نام Bean.
- 11 BeanClassLoaderAware: اینترفیس دریافت ClassLoader.

12• **BeanFactoryAware**: اینترفیس دریافت BeanFactory.

13• **setBeanName**: متد دریافت نام Bean.

14• **setBeanClassLoader**: متد دریافت ClassLoader.

15• **setBeanFactory**: متد دریافت BeanFactory.

## ۱۱.۲ تزریق وابستگی (DI)

1 • Constructor Injection: تزریق از طریق سازنده (توصیه شده).

2 • Setter Injection: تزریق از طریق setter.

3 • Field Injection: تزریق مستقیم روی فیلد (توصیه نشده).

4 • **Autowired@**: انوتیشن تزریق خودکار.

5 • **Qualifier@**: انتخاب Bean خاص در تزریق.

6 • **Primary: Bean@**: پیش فرض در تزریق.

7 • **Lazy@**: ساخت Bean با تأخیر.

8 • **Bean@**: تعریف Bean در کلاس Configuration.

9 • **Configuration@**: کلاس تعریف Bean ها.

10• **Import@**: وارد کردن کلاس های Configuration.

11• **Component@**: نشانه گذاری کلاس به عنوان Bean.

12• **Service: Component@**: لایه سرویس.

13• **Repository: Component@**: لایه دسترسی داده.

14• **Controller: Component@**: لایه کنترلر MVC.

15• **RestController@**: کنترلر REST با خروجی JSON.

16• **ResponseBody@**: بدنه پاسخ مستقیماً برگردانده می شود.

17• **RequestMapping@**: نگاشت مسیر درخواست.

18• **GetMapping@**: نگاشت GET.

19• **PostMapping@**: نگاشت POST.

20• **PutMapping@**: نگاشت PUT.

21• **DeleteMapping@**: نگاشت DELETE.

22• **PathVariable@**: استخراج متغیر از مسیر.

23• **RequestBody@**: استخراج بدنه درخواست.

24• **ModelAttribute@**: اتصال پارامترها به مدل.

### Transactional@ ۱۱.۳

1• **Transactional@**: مدیریت تراکنش به صورت اعلانی.

2• **Proxy**: واسط ایجادشده برای اعمال تراکنش.

3• **Transaction**: واحد کاری اتمیک.

4• **Commit**: تأیید نهایی تراکنش.

5• **Rollback**: بازگردانی تراکنش.

6• **RuntimeException**: باعث rollback پیش فرض.

7• **Checked Exception**: باعث rollback پیش فرض نمی شود.

8• **rollbackFor**: تعیین استثنای باعث rollback.

### Scope ۱۱.۴ها

1• **singleton**: یک نمونه در کل **ApplicationContext** (پیش فرض).

2• **prototype**: نمونه جدید در هر درخواست.

3• **request**: یک نمونه در هر HTTP request.

4• **session**: یک نمونه در هر HTTP session.

5• **application**: یک نمونه در کل **ServletContext**.

6• **Scope@**: تعیین scope برای Bean.

### Conditional Beans ۱۱.۵

1• **ConditionalOnClass@**: ساخت Bean در صورت وجود کلاس.

2• **ConditionalOnMissingClass@**: ساخت Bean در صورت نبود کلاس.

3• **ConditionalOnProperty@**: ساخت Bean بر اساس property.

4• **ConditionalOnBean@**: ساخت Bean در صورت وجود Bean دیگر.

5• **havingValue**: مقدار مورد انتظار property.

### تنظیمات ۱۱.۶

- 1 `application.properties`: فایل تنظیمات Spring Boot.
- 2 `server.port`: پورت سرور.
- 3 `server.servlet.context-path`: مسیر پایه اپلیکیشن.
- 4 `spring.datasource.url`: آدرس دیتابیس.
- 5 `spring.datasource.username`: نام کاربری دیتابیس.
- 6 `spring.datasource.password`: رمز دیتابیس.
- 7 `spring.jpa.hibernate.ddl-auto`: حالت ساخت/به‌روزرسانی `schema`.
- 8 `spring.jpa.show-sql`: نمایش کوئری‌های SQL.
- 9 `spring.jpa.properties.hibernate.dialect`: دیالکت دیتابیس.
- 10 `logging.level`: سطح لاگ‌گیری.

## ◆ بخش ۱۲: JPA، JDBC و Hibernate

### JDBC ۱۲.۱

- 1 API: JDBC (Java Database Connectivity) سطح‌پایین اتصال جاوا به دیتابیس.
- 2 `Driver`: درایور اختصاصی هر دیتابیس.
- 3 `Class.forName()`: بارگذاری کلاس درایور.
- 4 `DriverManager.getConnection()`: برقراری اتصال به دیتابیس.
- 5 `Connection`: نماینده اتصال به دیتابیس.
- 6 `Statement`: اجرای کوئری ساده.
- 7 `PreparedStatement`: اجرای کوئری پارامتری و امن.
- 8 `ResultSet`: نتیجه کوئری `SELECT`.
- 9 `executeQuery()`: اجرای کوئری `SELECT`.
- 10 `executeUpdate()`: اجرای `INSERT/UPDATE/DELETE`.
- 11 `rs.next()`: رفتن به ردیف بعدی `ResultSet`.
- 12 `rs.getString()`: خواندن ستون `String`.
- 13 `rs.getInt()`: خواندن ستون `int`.
- 14 `close()`: بستن منبع.

15• SQL Injection: تزریق SQL از ورودی.

16• concatenation: چسباندن رشته‌ها (خطر SQL Injection).

## JPA ۱۲.۲

1 • JPA (Java Persistence API): استاندارد ORM در جاوا.

2 • ORM: نگاشت آبجکت به جدول دیتابیس.

3 • Entity: کلاس نگاشت‌شده به جدول.

4 • Entity@: انوتیشن تعریف Entity.

5 • Table@: تعیین نام جدول.

6 • Id@: تعیین کلید اصلی.

7 • GeneratedValue@: تولید خودکار کلید.

8 • GenerationType.IDENTITY: تولید با auto-increment دیتابیس.

9 • Column@: تنظیم ستون.

10 • OneToMany@: رابطه یک‌به‌چند.

11 • ManyToOne@: رابطه چندبه‌یک.

12 • JoinColumn@: تعیین ستون کلید خارجی.

13 • mappedBy: سمت معکوس رابطه.

14 • cascade: انتشار عملیات به موجودیت‌های مرتبط.

15 • CascadeType.ALL: همه عملیات cascade.

16 • FetchType: نحوه بارگذاری رابطه.

17 • EAGER: بارگذاری فوری.

18 • LAZY: بارگذاری با تأخیر.

19 • N+1 Query Problem: اجرای N کوئری اضافی برای روابط.

20 • JOIN FETCH: بارگذاری رابطه با یک کوئری.

21 • EntityGraph: گراف بارگذاری موجودیت.

22 • NamedEntityGraph@: تعریف EntityGraph نام‌دار.

23 • NamedAttributeNode@: تعیین attribute در EntityGraph.

## Hibernate ۱۲.۳

- 1• **Hibernate**: پیاده‌سازی محبوب JPA.
- 2• **Session**: جلسه کار با دیتابیس در Hibernate.
- 3• **EntityManager**: اینترفیس JPA برای مدیریت موجودیت‌ها.
- 4• **Query**: کوئری JPQL.
- 5• **uniqueResult()**: دریافت نتیجه یکتا.
- 6• **createQuery()**: ساخت کوئری.
- 7• **setParameter()**: مقداردهی پارامتر کوئری.

#### Spring Data JPA ۱۲.۴

- 1• **Spring Data JPA**: لایه انتزاعی روی JPA.
- 2• **JpaRepository**: اینترفیس CRUD + امکانات پیشرفته.
- 3• **CrudRepository**: اینترفیس ساده CRUD.
- 4• **PagingAndSortingRepository**: اینترفیس صفحه‌بندی و مرتب‌سازی.
- 5• **findByUsername()**: جستجو بر اساس نام کاربری.
- 6• **findByUsernameAndPassword()**: جستجو با دو شرط.
- 7• **findByAgeGreaterThan()**: جستجو با شرط بزرگ‌تر.
- 8• **findByUsernameStartingWith()**: جستجو با شروع رشته.
- 9• **findAllOrderByUsernameAsc()**: همه با ترتیب صعودی.
- 10• **findAll(Pageable)**: صفحه‌بندی نتایج.
- 11• **existsByUsername()**: بررسی وجود.
- 12• **countByAgeGreaterThan()**: شمارش با شرط.
- 13• **deleteByUsername()**: حذف با شرط.
- 14• **Query@**: تعریف کوئری دلخواه.
- 15• **Param@**: نام‌گذاری پارامتر کوئری.
- 16• **nativeQuery**: استفاده از SQL خام.
- 17• **Modifying@**: برای کوئری‌های UPDATE/DELETE.
- 18• **Pageable**: اینترفیس صفحه‌بندی.
- 19• **PageRequest**: پیاده‌سازی Pageable.

- **Sort 20**: مرتب‌سازی.
- **Page 21**: نتیجه صفحه‌بندی‌شده.
- **Specification 22**: ساخت شرط‌های پویا.
- **Specification.where 23**(): شروع ساخت Specification.

## ◆ بخش ۱۳: DevOps، Docker، Kubernetes

### DevOps ۱۳.۱

- **DevOps 1**: فرهنگ همکاری توسعه و عملیات.
- **CI/CD 2**: یکپارچه‌سازی و تحویل/استقرار مداوم.
- **Continuous Integration 3**: ادغام مکرر کد در شاخه اصلی.
- **Continuous Deployment 4**: استقرار خودکار تا تولید.
- **Jenkins 5**: سرور CI/CD متن‌باز.
- **GitHub Actions: CI/CD 6**: داخلی گیت‌هاب.
- **GitLab CI: CI/CD 7**: داخلی گیت‌لب.
- **CircleCI 8**: سرویس CI/CD ابری.
- **Plan 9**: مرحله برنامه‌ریزی.
- **Code 10**: مرحله کدنویسی.
- **Build 11**: مرحله ساخت.
- **Test 12**: مرحله تست.
- **Release 13**: مرحله انتشار.
- **Deploy 14**: مرحله استقرار.
- **Operate 15**: مرحله بهره‌برداری.
- **Monitor 16**: مرحله پایش.

### Docker ۱۳.۲

- **Docker 1**: پلتفرم کانتینری‌سازی.
- **Container 2**: نمونه اجرایی سبک از ایمیج.

- **Image 3**: قالب فقطخواندنی برای ساخت کانتینر.
- **Dockerfile 4**: فایل دستورات ساخت ایمیج.
- **FROM 5**: تعیین ایمیج پایه.
- **ENV 6**: تعیین متغیر محیطی.
- **WORKDIR 7**: تعیین دایرکتوری کاری.
- **COPY 8**: کپی فایل به ایمیج.
- **EXPOSE 9**: اعلام پورت.
- **ENTRYPOINT 10**: دستور اجرای اصلی کانتینر.
- **docker build 11**: ساخت ایمیج.
- **docker run 12**: اجرای کانتینر.
- **docker ps 13**: لیست کانتینرهای در حال اجرا.
- **docker stop 14**: توقف کانتینر.
- **docker rm 15**: حذف کانتینر.
- **docker images 16**: لیست ایمیج‌ها.
- **docker exec 17**: اجرای دستور در کانتینر در حال اجرا.
- **docker logs 18**: مشاهده لاگ کانتینر.
- **Docker Compose 19**: ابزار مدیریت چند کانتینر.
- **docker-compose.yml 20**: فایل تعریف سرویس‌ها.
- **services 21**: بخش تعریف سرویس‌ها.
- **depends\_on 22**: وابستگی ترتیبی سرویس‌ها.
- **environment 23**: متغیرهای محیطی.
- **ports 24**: نگاشت پورت‌ها.
- **docker-compose up 25**: اجرای سرویس‌ها.
- **docker-compose down 26**: توقف و حذف سرویس‌ها.
- **Virtual Machine 27**: ماشین مجازی با OS کامل.
- **Host OS 28**: سیستم‌عامل میزبان.
- **Base Image 29**: ایمیج پایه.

- **1 Kubernetes**: ارکستراتور کانتینر.
- **2 K8s**: مخفف Kubernetes.
- **3 Pod**: کوچک‌ترین واحد اجرایی شامل یک یا چند کانتینر.
- **4 Deployment**: مدیریت نسخه‌ها و replica های Pod.
- **5 Service**: نقطه دسترسی پایدار به Pod ها.
- **6 Ingress**: مدیریت دسترسی HTTP/HTTPS خارجی.
- **7 ConfigMap**: نگهداری تنظیمات غیرحساس.
- **8 Secret**: نگهداری داده‌های حساس.
- **9 kubectl**: ابزار خط فرمان Kubernetes.
- **10 kubectl apply**: اعمال manifest.
- **11 kubectl get pods**: لیست Pod ها.
- **12 kubectl get services**: لیست سرویس‌ها.
- **13 kubectl get deployments**: لیست Deployment ها.
- **14 kubectl scale**: تغییر تعداد replica.
- **15 replicas**: تعداد نمونه‌های Pod.
- **16 LoadBalancer**: نوع سرویس با IP خارجی.
- **17 containerPort**: پورت داخلی کانتینر.
- **18 targetPort**: پورت مقصد در Pod.

#### Kafka ۱۳.۴

- **1 Apache Kafka**: پلتفرم streaming توزیع‌شده.
- **2 Producer**: تولیدکننده پیام.
- **3 Consumer**: مصرف‌کننده پیام.
- **4 Topic**: کانال پیام‌ها.
- **5 Partition**: بخش‌های موازی Topic.
- **6 Broker**: سرور Kafka.
- **7 KafkaTemplate**: کلاس ارسال پیام در Spring Kafka.
- **8 @KafkaListener**: انوتیشن مصرف پیام.

- **9 groupId**: شناسه گروه مصرف‌کننده.
- **10 Event Sourcing**: ذخیره رویدادها به جای وضعیت.
- **11 Log Aggregation**: جمع‌آوری لاگ‌ها.
- **12 Stream Processing**: پردازش جریان داده.
- **13 Message Queue**: صف پیام.

## Linux ۱۳.۵

- **1 ls**: لیست فایل‌ها.
- **2 cd**: تغییر دایرکتوری.
- **3 mkdir**: ساخت دایرکتوری.
- **4 rm**: حذف فایل/دایرکتوری.
- **5 cp**: کپی.
- **6 mv**: انتقال/تغییر نام.
- **7 cat**: نمایش محتوای فایل.
- **8 less**: نمایش صفحه‌به‌صفحه فایل.
- **9 tail**: نمایش انتهای فایل.
- **10 head**: نمایش ابتدای فایل.
- **11 ps**: لیست process‌ها.
- **12 top**: نمایش زنده process‌ها.
- **13 kill**: ارسال سیگنال به process.
- **14 nohup**: اجرای process مستقل از terminal.
- **15 ping**: بررسی اتصال شبکه.
- **16 netstat**: نمایش اتصالات شبکه.
- **17 curl**: ارسال درخواست HTTP.
- **18 ss**: نمایش سوکت‌ها.
- **19 df**: فضای دیسک.
- **20 du**: حجم دایرکتوری.
- **21 free**: وضعیت حافظه.

- 22• apt-get: مدیریت بسته Debian/Ubuntu.
- 23• yum: مدیریت بسته RedHat/CentOS.
- 24• Debian: توزیع پایه اوبونتو.
- 25• Ubuntu: توزیع محبوب Debian-based.
- 26• RedHat: توزیع تجاری لینوکس.
- 27• CentOS: توزیع متن‌باز RedHat-based.
- 28• APT: سیستم مدیریت بسته Debian.
- 29• RPM: سیستم مدیریت بسته RedHat.

## ♦ بخش ۱۴: تست‌نویسی (Testing)

### ۱۴.۱ مفاهیم پایه

- 1• Unit Test: تست یک واحد کوچک کد.
- 2• Integration Test: تست تعامل چند ماژول.
- 3• E2E Test: تست کامل جریان از ابتدا تا انتها.
- 4• Test Pyramid: هرم **تست** (unit زیاد، integration متوسط، E2E کم).
- 5• FIRST: اصول تست خوب.
- 6• Fast: سریع.
- 7• Independent: مستقل از سایر تست‌ها.
- 8• Repeatable: تکرارپذیر.
- 9• Self-Validating: خوداعتبارسنج.
- 10• Timely: به‌موقع.
- 11• Given-When-Then: ساختار **تست** (شرط-عمل-نتیجه).
- 12• Arrange-Act-Assert: چیدمان-عمل-بررسی.
- 13• AAA: مخفف Arrange-Act-Assert.
- 14• GWT: مخفف Given-When-Then.
- 15• Boundary Value Analysis: تست مقادیر مرزی.
- 16• Equivalence Partitioning: تقسیم ورودی‌ها به کلاس‌های هم‌ارز.

- 17• Negative Testing: تست ورودی‌های نامعتبر.
- 18• Regression Bug: باگ بازگشتی پس از تغییر.
- 19• TDD: توسعه مبتنی بر تست.
- 20• Red-Green-Refactor: چرخه TDD.
- 21• Code Coverage: درصد کد پوشش داده شده.
- 22• Line Coverage: پوشش خطوط.
- 23• Branch Coverage: پوشش شاخه‌ها.
- 24• Method Coverage: پوشش متدها.
- 25• JaCoCo: ابزار سنجش پوشش کد.

#### JUnit 5 ۱۴.۲

- 1 • JUnit 5: فریمورک تست جاوا.
- 2 • Test@: انوتیشن متد تست.
- 3 • BeforeAll@: اجرا یک بار قبل از همه تست‌ها.
- 4 • AfterAll@: اجرا یک بار بعد از همه تست‌ها.
- 5 • BeforeEach@: اجرا قبل از هر تست.
- 6 • AfterEach@: اجرا بعد از هر تست.
- 7 • DisplayName@: نام نمایشی تست.
- 8 • ParameterizedTest@: تست پارامتری.
- 9 • ValueSource@: منبع مقادیر ساده.
- 10 • CsvSource@: منبع مقادیر CSV.
- 11 • EnumSource@: منبع مقادیر enum.
- 12 • assertEquals: بررسی برابری.
- 13 • assertNotEquals: بررسی عدم برابری.
- 14 • assertNull: بررسی null بودن.
- 15 • assertNotNull: بررسی null نبودن.
- 16 • assertTrue: بررسی true بودن.
- 17 • assertFalse: بررسی false بودن.

- 18• **assertThrows**: بررسی پرتاب استثنا.
- 19• **assertAll**: گروه‌بندی assertion ها.
- 20• **assertArrayEquals**: بررسی برابری آرایه‌ها.
- 21• **assertTimeout**: بررسی محدودیت زمانی.
- 22• **Duration**: کلاس نمایش مدت زمان.

#### Mockito ۱۴.۳

- 1• **Mockito**: فریمورک mock سازی.
- 2• **Mock**: آجکت جعلی.
- 3• **Stub**: پاسخ از پیش تعیین شده.
- 4• **Mock@**: انوتیشن ساخت mock.
- 5• **InjectMocks@**: تزریق mock ها به کلاس هدف.
- 6• **Spy@**: جاسوس روی آجکت واقعی.
- 7• **when()**: تعریف رفتار mock.
- 8• **thenReturn()**: تعیین مقدار بازگشتی.
- 9• **thenThrow()**: تعیین استثنای پرتابی.
- 10• **doThrow()**: پرتاب استثنا در متد **void**.
- 11• **verify()**: بررسی فراخوانی متد.
- 12• **times()**: تعداد فراخوانی.
- 13• **never()**: هرگز فراخوانی نشده.
- 14• **atLeast()**: حداقل تعداد فراخوانی.
- 15• **atMost()**: حداکثر تعداد فراخوانی.
- 16• **verifyNoMoreInteractions()**: بررسی عدم فراخوانی اضافه.
- 17• **any()**: هر آرگومان.
- 18• **eq()**: آرگومان مشخص.
- 19• **anyString()**: هر String.
- 20• **anyLong()**: هر **long**.
- 21• **MockitoAnnotations.openMocks()**: فعال‌سازی انوتیشن‌ها.

## Spring Boot Testing ۱۴.۴

- 1 **SpringBootTest@**: تست کامل Spring context.
- 2 **WebMvcTest@**: تست لایه کنترلر.
- 3 **DataJpaTest@**: تست لایه JPA.
- 4 **MockBean@**: جایگزینی Bean با mock.
- 5 **MockMvc**: شبیه ساز درخواست HTTP.
- 6 **TestRestTemplate**: کلاینت REST برای تست.
- 7 **LocalServerPort@**: دریافت پورت تصادفی.
- 8 **webEnvironment**: تنظیم محیط وب تست.
- 9 **RANDOM\_PORT**: پورت تصادفی.
- 10 **MockMvcRequestBuilders**: ساخت درخواست تست.
- 11 **MockMvcResultMatchers**: بررسی پاسخ تست.
- 12 **jsonPath()**: بررسی مقدار JSON.
- 13 **status()**: بررسی کد وضعیت.
- 14 **contentType()**: بررسی نوع محتوا.
- 15 **MediaType.APPLICATION\_JSON**: نوع JSON.
- 16 **HttpStatus**: کدهای وضعیت HTTP.
- 17 **ResponseEntity**: پاسخ HTTP.

## ◆ بخش ۱۵: JSP و Thymeleaf

## Thymeleaf ۱۵.۱

- 1 **Thymeleaf**: قالب ساز سمت سرور Spring.
- 2 **Template Engine**: موتور پردازش قالب.
- 3 **Natural Templating**: قالب قابل نمایش در مرورگر بدون پردازش.
- 4 **spring-boot-starter-thymeleaf**: استارتر Thymeleaf در Spring Boot.
- 5 **spring.thymeleaf.prefix**: پیشوند مسیر قالب ها.
- 6 **spring.thymeleaf.suffix**: پسوند فایل قالب.

- 7 `spring.thymeleaf.cache`: فعال/غیرفعال کردن کش قالب.
- 8 `spring.thymeleaf.encoding`: کدگذاری قالب.
- 9 `Model`: ظرف داده‌های ارسالی به `view`.
- 10 `model.addAttribute()`: افزودن داده به `Model`.
- 11 `Controller@`: کنترلر MVC.
- 12 `GetMapping@`: نگاشت GET.
- 13 `PostMapping@`: نگاشت POST.
- 14 `ModelAttribute@`: اتصال داده به مدل.
- 15 `redirect::`: ریدایرکت به مسیر دیگر.
- 16 `th:text`: درج متن.
- 17 `th:if`: شرط نمایش.
- 18 `th:unless`: شرط معکوس نمایش.
- 19 `th:each`: حلقه روی مجموعه.
- 20 `th:href`: لینک.
- 21 `th:action`: آدرس فرم.
- 22 `th:object`: شیء فرم.
- 23 `th:field`: اتصال فیلد فرم.
- 24 `th:errors`: نمایش خطاهای اعتبارسنجی.
- 25 `th:include`: درج fragment.
- 26 `th:replace`: جایگزینی fragment.
- 27 `th:fragment`: تعریف fragment.
- 28 `fields.hasErrors()`: بررسی خطای فیلد.
- 29 `i18n`: بین‌المللی‌سازی.
- 30 `messages_fa.properties`: فایل پیام فارسی.
- 31 `messages_en.properties`: فایل پیام انگلیسی.
- 32 `sec:authorize`: کنترل دسترسی در قالب.
- 33 `sec:authentication`: دسترسی به اطلاعات احراز هویت.
- 34 `isAuthenticated()`: بررسی ورود کاربر.

35 • `hasRole('ADMIN')`: بررسی نقش.

36 • `thymeleaf-extras-springsecurity6`: افزونه امنیت Thymeleaf.

## JSP ۱۵.۲

- 1 • JSP (JavaServer Pages): فناوری قدیمی قالب‌ساز جاوا.
- 2 • `tomcat-embed-jasper`: موتور JSP برای Tomcat داخلی.
- 3 • `jstl`: کتابخانه تگ استاندارد JSP.
- 4 • `spring.mvc.view.prefix`: پیشوند مسیر `view`.
- 5 • `spring.mvc.view.suffix`: پسوند فایل `view`.
- 6 • **JSTL**: JavaServer Pages Standard Tag Library.
- 7 • `c:if`: شرط در JSTL.
- 8 • `c:forEach`: حلقه در JSTL.
- 9 • `c:out`: چاپ مقدار.
- 10 • `fmt`: تگ‌های قالب‌بندی.
- 11 • Expression Language: `{ } $`.
- 12 • `<% page @%>`: دایرکتیو صفحه.
- 13 • `<% taglib @%>`: دایرکتیو کتابخانه تگ.
- 14 • `/WEB-INF/views`: مسیر معمول فایل‌های JSP.

## بخش ۱۶: Gradle و Maven

### Maven ۱۶.۱

- 1 • **Maven**: ابزار مدیریت `build` و وابستگی جاوا.
- 2 • `pom.xml`: فایل اصلی پیکربندی Maven.
- 3 • `groupId`: شناسه گروه/سازمان.
- 4 • `artifactId`: شناسه پروژه.
- 5 • `version`: نسخه پروژه.
- 6 • `packaging`: نوع بسته‌بندی (`jar/war`).

- **parent 7**: پروژه والد ارث‌بری.
- **properties 8**: تعریف property‌های قابل استفاده.
- **dependencies 9**: بخش تعریف وابستگی‌ها.
- **dependency 10**: یک وابستگی مشخص.
- **scope 11**: دامنه وابستگی.
- **compile: scope 12**: پیش‌فرض.
- **provided 13**: وابستگی فراهم‌شده در زمان اجرا توسط container.
- **runtime 14**: وابستگی موردنیاز فقط زمان اجرا.
- **test 15**: وابستگی فقط برای تست.
- **system 16**: وابستگی سیستمی (منسوخ).
- **build 17**: بخش پیکربندی build.
- **plugins 18**: پلاگین‌های build.
- **plugin 19**: یک پلاگین مشخص.
- **repositories 20**: مخازن دانلود وابستگی.
- **mvn clean 21**: پاک کردن خروجی build.
- **mvn compile 22**: کامپایل کد.
- **mvn test 23**: اجرای تست‌ها.
- **mvn package 24**: ساخت بسته (jar/war).
- **mvn install 25**: نصب در مخزن محلی.
- **mvn spring-boot:run 26**: اجرای اپلیکیشن Spring Boot.
- **mvn dependency:tree 27**: نمایش درخت وابستگی‌ها.
- **spring-boot-maven-plugin 28**: پلاگین بسته‌بندی Spring Boot.
- **maven-compiler-plugin 29**: پلاگین کامپایلر.
- **jacoco-maven-plugin 30**: پلاگین سنجش پوشش کد.
- **Lifecycle 31**: چرخه حیات build.
- **validate 32**: اعتبارسنجی پروژه.
- **deploy 33**: انتشار در مخزن راه دور.
- **site 34**: تولید مستندات سایت.

- **1 Gradle**: ابزار build مدرن و انعطاف‌پذیر.
- **2 build.gradle**: فایل build پروژه.
- **3 settings.gradle**: فایل تنظیمات پروژه.
- **4 plugins**: افزونه‌های build.
- **5 group**: شناسه گروه پروژه.
- **6 version**: نسخه پروژه.
- **7 sourceCompatibility**: نسخه سازگاری سورس.
- **8 repositories**: مخازن دانلود وابستگی.
- **9 mavenCentral**(**mavenCentral**): مخزن مرکزی Maven.
- **10 dependencies**: بخش وابستگی‌ها.
- **11 implementation**: وابستگی اصلی.
- **12 runtimeOnly**: وابستگی فقط زمان اجرا.
- **13 compileOnly**: وابستگی فقط زمان کامپایل.
- **14 annotationProcessor**: پردازنده انوتیشن.
- **15 testImplementation**: وابستگی تست.
- **16 testRuntimeOnly**: وابستگی تست زمان اجرا.
- **17 useJUnitPlatform**(**useJUnitPlatform**): استفاده از JUnit Platform.
- **18 task**: تعریف task سفارشی.
- **19 doLast**: اجرای عمل در پایان task.
- **20 gradlew tasks/**: لیست task‌ها.
- **21 gradlew clean/**: پاک کردن build.
- **22 gradlew build/**: ساخت پروژه.
- **23 gradlew bootRun/**: اجرای Spring Boot.
- **24 gradlew dependencies/**: نمایش وابستگی‌ها.
- **25 Incremental Build**: ساخت افزایشی برای سرعت بیشتر.

## ◆ بخش ۱۷: میکروسرویس‌ها (Microservices)

### ۱۷.۱ مفاهیم پایه

- 1 **Microservices**: معماری تقسیم برنامه به سرویس‌های کوچک مستقل.
- 2 **Monolith**: برنامه یکپارچه با یک واحد استقرار.
- 3 **Business Capability**: تقسیم سرویس‌ها بر اساس قابلیت کسب‌وکار.
- 4 **Independent Deploy**: استقرار مستقل هر سرویس.
- 5 **HTTP**: پروتکل ارتباط بین سرویس‌ها.
- 6 **Message Queue**: صف پیام برای ارتباط غیرهم‌زمان.
- 7 **Scalability**: مقیاس‌پذیری.
- 8 **Latency**: تأخیر.
- 9 **Distributed Transactions**: تراکنش‌های توزیع‌شده.
- 10 **Service Discovery**: کشف خودکار سرویس‌ها.
- 11 **API Gateway**: دروازه ورودی به میکروسرویس‌ها.
- 12 **Rate Limiting**: محدودسازی نرخ درخواست.
- 13 **Circuit Breaker**: قطع‌کننده مدار برای جلوگیری از خرابی آبشاری.
- 14 **Load Balancer**: توزیع‌کننده بار.
- 15 **Distributed Tracing**: ردیابی درخواست‌ها در سرویس‌های مختلف.

### ۱۷.۲ Spring Cloud

- 1 **Spring Cloud**: مجموعه ابزار میکروسرویس در Spring.
- 2 **Spring Cloud Netflix**: مجموعه ابزار Netflix.
- 3 **Eureka**: سرور کشف سرویس.
- 4 **Eureka Server**: سرور ثبت سرویس‌ها.
- 5 **Eureka Client**: کلاینت ثبت/کشف سرویس.
- 6 **EnableEurekaServer@**: فعال‌سازی سرور Eureka.
- 7 **EnableDiscoveryClient@**: فعال‌سازی کشف سرویس.

- 8 **EnableCircuitBreaker@**: فعال سازی Circuit Breaker.
- 9 **spring-cloud-starter-netflix-eureka-server**: استارتر Eureka Server.
- 10 **spring-cloud-starter-netflix-eureka-client**: استارتر Eureka Client.
- 11 **spring-cloud-starter-gateway**: استارتر API Gateway.
- 12 **spring-cloud-starter-circuitbreaker-resilience4j**: استارتر Resilience4j.
- 13 **spring-cloud-starter-loadbalancer**: استارتر LoadBalancer.
- 14 **spring-cloud-dependencies**: مدیریت نسخه Spring Cloud.
- 15 **eureka.client.register-with-eureka**: ثبت خود در Eureka.
- 16 **eureka.client.fetch-registry**: دریافت رجیستری.
- 17 **eureka.client.serviceUrl.defaultZone**: آدرس Eureka.
- 18 **eureka.instance.prefer-ip-address**: ترجیح IP به hostname.
- 19 **eureka.instance.instance-id**: شناسه نمونه.
- 20 **LoadBalancerClient**: کلاینت توزیع بار.
- 21 **RestTemplate**: کلاینت HTTP ساده.
- 22 **LoadBalanced@**: فعال سازی LoadBalancer روی RestTemplate.
- 23 **DiscoveryClient**: کلاینت کشف سرویس.
- 24 **ServiceInstance**: نمونه سرویس کشف شده.
- 25 **lb://user-service**: آدرس منطقی سرویس با *LoadBalancer*.

### ۱۷.۳ API Gateway

- 1 **API Gateway**: دروازه ورودی سرویس ها.
- 2 **spring.cloud.gateway.routes**: تعریف route ها.
- 3 **id**: شناسه route.
- 4 **uri**: مقصد route.
- 5 **predicates**: شرط های تطبیق route.
- 6 **Path: predicate**: تطبیق مسیر.
- 7 **filters**: فیلترهای route.
- 8 **CircuitBreaker**: فیلتر Circuit Breaker.

- **9 fallbackUri**: مسیر جایگزین در خطا.
- **10 Retry**: فیلتر تلاش مجدد.
- **11 GlobalFilter**: فیلتر سراسری Gateway.
- **12 ServerHttpRequest**: درخواست HTTP در WebFlux.
- **13 ServerHttpResponse**: پاسخ HTTP در WebFlux.
- **14 RedisRateLimiter**: محدودکننده نرخ مبتنی بر Redis.

#### Circuit Breaker ۱۷.۴

- **1 Resilience4j**: کتابخانه تحمل خطا.
- **2 CircuitBreaker@**: انوتیشن Circuit Breaker.
- **3 fallbackMethod**: متد جایگزین در خطا.
- **4 Retry@**: تلاش مجدد خودکار.
- **5 RateLimiter@**: محدودسازی نرخ.
- **6 Bulkhead@**: محدودسازی همزمانی.
- **7 sliding-window-size**: اندازه پنجره لغزان.
- **8 failure-rate-threshold**: آستانه نرخ خطا.
- **9 wait-duration-in-open-state**: مدت باز ماندن مدار.
- **10 minimum-number-of-calls**: حداقل تعداد فراخوانی.
- **11 max-attempts**: حداکثر تلاش.
- **12 wait-duration**: مدت انتظار بین تلاش‌ها.
- **13 limit-for-period**: محدودیت در بازه.
- **14 limit-refresh-period**: بازه بازنشانی محدودیت.
- **15 max-concurrent-calls**: حداکثر فراخوانی همزمان.

#### Distributed Tracing ۱۷.۵

- **1 Sleuth**: ابزار ردیابی Spring Cloud.
- **2 Zipkin**: سرور نمایش trace.
- **3 spring-cloud-starter-sleuth**: استارتر Sleuth.

- 4• `spring-cloud-starter-zipkin`: استارتر Zipkin.
- 5• `spring.sleuth.sampler.probability`: احتمال نمونه‌گیری.
- 6• `spring.zipkin.base-url`: آدرس Zipkin.
- 7• `Tracer`: مؤلفه ساخت `span`.
- 8• `Span`: واحد کار در `trace`.
- 9• `Trace ID`: شناسه یکتای `trace`.

## ◆ بخش ۱۸: اصطلاحات عمومی جاوا

### ۱۸.۱ محیط و ابزارها

- 1• `JDK`: کیت توسعه جاوا شامل کامپایلر و `JRE`.
- 2• `JRE`: محیط اجرای جاوا شامل `JVM` و کتابخانه‌ها.
- 3• `JVM`: ماشین مجازی اجرای بایت‌کد جاوا.
- 4• `Java SE`: نسخه استاندارد جاوا.
- 5• `Java EE`: نسخه سازمانی **جاوا** (اکنون `Jakarta EE`).
- 6• `Java ME`: نسخه سبک جاوا برای دستگاه‌های محدود.
- 7• `javac`: کامپایلر جاوا.
- 8• `java`: دستور اجرای برنامه جاوا.
- 9• `jar`: ابزار ساخت/مدیریت فایل `JAR`.
- 10• `javadoc`: ابزار تولید مستندات.
- 11• `JAR`: آرشیو جاوا.
- 12• `WAR`: آرشیو وب اپلیکیشن جاوا.
- 13• `Classpath`: مسیر جستجوی کلاس‌ها.
- 14• `ClassLoader`: بارگذار کلاس‌ها.
- 15• `Bytecode`: کد میانی اجراشده توسط `JVM`.
- 16• `Garbage Collector`: جمع‌آورنده خودکار حافظه.
- 17• `JIT` (Just-In-Time): کامپایلر بلادرنگ بایت‌کد به ماشین.
- 18• `JVM Memory Model`: مدل حافظه `JVM`.

## ۱۸.۲ اصطلاحات عمومی

- 1 API: رابط برنامه‌نویسی.
- 2 Servlet: کامپوننت وب جاوا.
- 3 ServletContext: بستر مشترک اپلیکیشن وب.
- 4 Tomcat: کانتینر Servlet.
- 5 PostgreSQL: دیتابیس رابطه‌ای متن‌باز.
- 6 MySQL: دیتابیس رابطه‌ای محبوب.
- 7 H2: دیتابیس درون‌حافظه‌ای جاوا.
- 8 Spring Framework: فریمورک جامع جاوا.
- 9 Spring Core: هسته (IoC/DI) Spring.
- 10 Spring Boot: فریمورک ساده‌سازی Spring.
- 11 Spring MVC: ماژول وب Spring.
- 12 Spring Data: ماژول دسترسی داده.
- 13 Spring Security: ماژول امنیت.
- 14 Spring Cloud: ماژول میکروسرویس.
- 15 IoC (Inversion of Control): وارونگی کنترل.
- 16 DI (Dependency Injection): تزریق وابستگی.
- 17 AOP (Aspect-Oriented Programming): برنامه‌نویسی جنبه‌گرا.
- 18 Bean: آبجکت مدیریت‌شده Spring.
- 19 ApplicationContext: context اصلی Spring.
- 20 BeanFactory: کارخانه پایه Bean.
- 21 DispatcherServlet: کنترلر جلویی Spring MVC.
- 22 Front Controller: الگوی کنترلر مرکزی.
- 23 ViewResolver: تشخیص view.
- 24 HandlerMapping: نگاشت درخواست به handler.
- 25 ModelAndView: مدل + view در Spring MVC.

## ۱۸.۳ اصطلاحات شبکه و وب

- 1 HTTP: پروتکل انتقال ابرمتن.
- 2 HTTPS: HTTP امن.
- 3 TCP: پروتکل انتقال مطمئن.
- 4 UDP: پروتکل انتقال سریع.
- 5 IP: پروتکل اینترنت.
- 6 DNS: سیستم نام دامنه.
- 7 REST: سبک معماری مبتنی بر منابع.
- 8 RESTful: سازگار با REST.
- 9 JSON: فرمت داده سبک.
- 10 XML: فرمت داده نشانه‌گذاری.
- 11 SOAP: پروتکل پیام‌رسانی مبتنی بر XML.
- 12 WebSocket: ارتباط دوطرفه پایدار.
- 13 CORS: اشتراک منابع بین دامنه‌ای.
- 14 CSRF: جعل درخواست از سایت دیگر.
- 15 XSS: تزریق اسکریپت.
- 16 SQL Injection: تزریق SQL.
- 17 JWT: توکن وب JSON.
- 18 OAuth: پروتکل مجوزدهی.
- 19 OAuth2: نسخه دوم OAuth.
- 20 Basic Auth: احراز هویت پایه.
- 21 Bearer Token: توکن حامل.
- 22 Session: نشست کاربر.
- 23 Cookie: داده ذخیره‌شده در مرورگر.
- 24 Local Storage: ذخیره‌سازی محلی مرورگر.
- 25 SSL/TLS: پروتکل امنیت ارتباط.
- 26 Certificate: گواهی دیجیتال.
- 27 Load Balancer: توزیع‌کننده بار.
- 28 Reverse Proxy: پروکسی معکوس.

- 29• Forward Proxy: پروکسی رو به جلو.
- 30• Nginx: وب سرور/پروکسی محبوب.
- 31• Apache: وب سرور محبوب.

#### ۱۸.۴ اصطلاحات دیتابیس

- 1• JDBC: API اتصال جاوا به دیتابیس.
- 2• JPA: استاندارد ORM جاوا.
- 3• Hibernate: پیاده سازی JPA.
- 4• ORM: نگاشت شیء-رابطه ای.
- 5• Entity: کلاس نگاشت شده به جدول.
- 6• Repository: لایه دسترسی داده.
- 7• DAO: شیء دسترسی داده.
- 8• DTO: شیء انتقال داده.
- 9• Transaction: واحد کاری اتمیک.
- 10• ACID: اتمی بودن، سازگاری، انزوا، پایداری.
- 11• Isolation Level: سطح انزوا تراکنش.
- 12• Lock: قفل.
- 13• Optimistic Lock: قفل خوش بینانه.
- 14• Pessimistic Lock: قفل بدبینانه.
- 15• Connection Pool: استخر اتصال.
- 16• HikariCP: استخر اتصال سریع.
- 17• Flyway: ابزار مهاجرت دیتابیس.
- 18• Liquibase: ابزار مهاجرت دیتابیس.
- 19• Migration: مهاجرت schema.
- 20• Schema: ساختار دیتابیس.
- 21• Index: فهرست برای سرعت جستجو.
- 22• Primary Key: کلید اصلی.
- 23• Foreign Key: کلید خارجی.

- 24• **Join**: اتصال جدول‌ها.
- 25• **Inner Join**: اتصال داخلی.
- 26• **Left Join**: اتصال چپ.
- 27• **Right Join**: اتصال راست.
- 28• **Subquery**: پرس‌وجوی تودرتو.
- 29• **View**: نمای مجازی.
- 30• **Stored Procedure**: رویه ذخیره‌شده.
- 31• **Trigger**: تریگر دیتابیس.

#### ۱۸.۵ اصطلاحات هم‌روندی

- 1 • **Thread**: ریسه اجرا.
- 2 • **Runnable**: اینترفیس task.
- 3 • **Callable**: اینترفیس task با خروجی.
- 4 • **ExecutorService**: مدیریت thread pool.
- 5 • **Thread Pool**: استخر thread.
- 6 • **Fork/Join**: چارچوب موازی‌سازی.
- 7 • **CompletableFuture**: API async جاوا ۸.
- 8 • **Future**: نتیجه آینده async.
- 9 • **AtomicInteger**: int اتمیک.
- 10 • **AtomicLong**: long اتمیک.
- 11 • **Volatile**: تضمین visibility متغیر.
- 12 • **Synchronized**: قفل هم‌زمانی.
- 13 • **Lock**: قفل پیشرفته.
- 14 • **ReentrantLock**: قفل قابل ورود مجدد.
- 15 • **ReadWriteLock**: قفل خواندن/نوشتن.
- 16 • **Semaphore**: سمافور.
- 17 • **CountDownLatch**: هماهنگ‌کننده انتظار.
- 18 • **CyclicBarrier**: مانع چرخه‌ای.

- 19• **Phaser**: هماهنگ‌کننده چندفازی.
- 20• **ConcurrentHashMap: Map**: هم‌زمان امن.
- 21• **CopyOnWriteArrayList**: لیست امن با کپی.
- 22• **BlockingQueue**: صف مسدودکننده.
- 23• **ArrayBlockingQueue**: صف مسدودکننده آرایه‌ای.
- 24• **LinkedBlockingQueue**: صف مسدودکننده پیوندی.
- 25• **PriorityBlockingQueue**: صف مسدودکننده اولویت‌دار.
- 26• **SynchronousQueue**: صف هم‌زمان.
- 27• **Deadlock**: بن‌بست.
- 28• **Livelock**: قفل زنده (تغییر بدون پیشرفت).
- 29• **Starvation**: گرسنگی thread.
- 30• **Race Condition**: شرایط رقابتی.
- 31• **Thread Safety**: امنیت thread.
- 32• **Happens-Before**: رابطه ترتیب حافظه.
- 33• **Memory Visibility**: دیدپذیری حافظه.

#### ۱۸.۶ اصطلاحات معماری

- 1 • **Monolith**: یکپارچه.
- 2 • **Microservices**: ریزسرویس‌ها.
- 3 • **SOA**: معماری سرویس‌گرا.
- 4 • **Event-Driven**: رویدادمحور.
- 5 • **Serverless**: بدون سرور.
- 6 • **Layered Architecture**: معماری لایه‌ای.
- 7 • **Hexagonal Architecture**: معماری شش‌ضلعی.
- 8 • **Clean Architecture**: معماری تمیز.
- 9 • **MVC**: مدل-نما-کنترلر.
- 10• **MVP**: مدل-نما-ارائه‌دهنده.
- 11• **MVVM**: مدل-نما-مدل‌نما.

- 12• **CQRS**: جداسازی خواندن/نوشتن.
- 13• **Event Sourcing**: ذخیره رویدادمحور.
- 14• **Saga Pattern**: الگوی Saga برای تراکنش توزیع شده.
- 15• **API Gateway**: دروازه API.
- 16• **Service Mesh**: مش سرویس.
- 17• **Sidecar**: کانتینر کمکی.
- 18• **Circuit Breaker**: قطع کننده مدار.
- 19• **Bulkhead**: جداسازی منابع.
- 20• **Retry**: تلاش مجدد.
- 21• **Timeout**: مهلت.
- 22• **Rate Limiting**: محدودسازی نرخ.
- 23• **Load Balancing**: توزیع بار.
- 24• **Service Discovery**: کشف سرویس.
- 25• **Config Server**: سرور تنظیمات.
- 26• **Distributed Tracing**: ردیابی توزیع شده.
- 27• **Centralized Logging**: لاگ متمرکز.
- 28• **Health Check**: بررسی سلامت.
- 29• **Graceful Shutdown**: خاموشی نرم.

#### ۱۸.۷ دیزاین پترن های تکمیلی

- 1 • **Repository Pattern**: لایه دسترسی داده.
- 2 • **DTO Pattern**: انتقال داده بین لایه ها.
- 3 • **DAO Pattern**: دسترسی داده.
- 4 • **Service Layer**: لایه منطق کسب و کار.
- 5 • **Controller**: کنترلر.
- 6 • **Facade**: فاساد.
- 7 • **Adapter**: آداپتور.
- 8 • **Decorator**: دکوراتور.

- 9 Proxy: پروکسی.
- 10 Factory: کارخانه.
- 11 Singleton: تک نمونه.
- 12 Builder: سازنده.
- 13 Observer: ناظر.
- 14 Strategy: استراتژی.
- 15 Template Method: متد قالب.
- 16 Chain of Responsibility: زنجیره مسئولیت.
- 17 Command: فرمان.
- 18 State: وضعیت.
- 19 Visitor: بازدیدکننده.
- 20 Mediator: واسطه.
- 21 Memento: یادگار.
- 22 Iterator: پیمایشگر.
- 23 Composite: ترکیبی.
- 24 Bridge: پل.
- 25 Flyweight: وزن سبک.
- 26 Abstract Factory: کارخانه انتزاعی.
- 27 Prototype: نمونه اولیه.

## ◆ بخش ۱۹: Annotation های مهم جاوا و Spring

### ۱۹.۱ Annotation های Core Java

- 1 @Override: بازنویسی متد والد.
- 2 @Deprecated: منسوخ شده.
- 3 @SuppressWarnings: سرکوب هشدارهای کامپایلر.
- 4 @SafeVarargs: تضمین امنیت متد varargs جنریک.
- 5 @FunctionalInterface: اینترفیس تک متدی.

- 6 @Retention: تعیین مدت نگهداری انوتیشن.
- 7 @Target: تعیین محل مجاز انوتیشن.
- 8 @Documented: درج در مستندات.
- 9 @Inherited: ارث‌بری انوتیشن توسط زیرکلاس.
- 10 @Repeatable: امکان تکرار انوتیشن.
- 11 @Native: نشانه‌گذاری فیلد بومی.

## ۱۹.۲ Annotation های Spring

- 1 @Component: Bean عمومی Spring.
- 2 @Service: Bean لایه سرویس.
- 3 @Repository: Bean لایه دسترسی داده.
- 4 @Controller: Bean کنترلر MVC.
- 5 @RestController: Bean کنترلر REST.
- 6 @Configuration: کلاس پیکربندی Bean ها.
- 7 @Bean: تعریف Bean در متد.
- 8 @Autowired: تزریق خودکار وابستگی.
- 9 @Qualifier: انتخاب Bean مشخص.
- 10 @Primary: پیش فرض.
- 11 @Lazy: ساخت با تأخیر.
- 12 @Value: تزریق مقدار از property.
- 13 @PropertySource: تعیین فایل property.
- 14 @Profile: فعال سازی شرطی بر اساس profile.
- 15 @Scope: تعیین scope.
- 16 @PostConstruct: بعد از ساخت Bean.
- 17 @PreDestroy: قبل از نابودی Bean.
- 18 @Import: وارد کردن Configuration.
- 19 @ImportResource: وارد کردن XML configuration.
- 20 @ComponentScan: اسکن پکیج برای Component ها.

- 21• **EnableAutoConfiguration@**: فعال سازی پیکربندی خودکار.
- 22• **SpringBootApplication@**: ترکیب سه انوتیشن اصلی Spring Boot.
- 23• **EnableWebSecurity@**: فعال سازی امنیت وب.
- 24• **EnableMethodSecurity@**: فعال سازی امنیت سطح متد.
- 25• **EnableEurekaServer@**: فعال سازی سرور Eureka.
- 26• **EnableDiscoveryClient@**: فعال سازی کشف سرویس.
- 27• **EnableCircuitBreaker@**: فعال سازی Circuit Breaker.
- 28• **EnableJpaRepositories@**: فعال سازی ریپازیتوری های JPA.
- 29• **EnableTransactionManagement@**: فعال سازی مدیریت تراکنش.
- 30• **EnableScheduling@**: فعال سازی زمان بندی.
- 31• **EnableAsync@**: فعال سازی async.
- 32• **EnableCaching@**: فعال سازی کش.
- 33• **ConditionalOnClass@**: شرط وجود کلاس.
- 34• **ConditionalOnMissingClass@**: شرط نبود کلاس.
- 35• **ConditionalOnProperty@**: شرط property.
- 36• **ConditionalOnBean@**: شرط وجود Bean.
- 37• **ConditionalOnMissingBean@**: شرط نبود Bean.

### Spring MVC ۱۹.۳ Annotation های

- 1 • **RequestMapping@**: نگاشت مسیر.
- 2 • **GetMapping@**: نگاشت GET.
- 3 • **PostMapping@**: نگاشت POST.
- 4 • **PutMapping@**: نگاشت PUT.
- 5 • **DeleteMapping@**: نگاشت DELETE.
- 6 • **PatchMapping@**: نگاشت PATCH.
- 7 • **PathVariable@**: متغیر مسیر.
- 8 • **RequestParam@**: پارامتر درخواست.
- 9 • **RequestBody@**: بدنه درخواست.

- 10• **ResponseBody@**: بدنه پاسخ.
- 11• **ModelAttribute@**: اتصال مدل.
- 12• **RequestHeader@**: هدر درخواست.
- 13• **CookieValue@**: مقدار کوکی.
- 14• **SessionAttribute: attribute@**: نشست.
- 15• **RequestAttribute: attribute@**: درخواست.
- 16• **CrossOrigin@**: فعال سازی CORS.
- 17• **ExceptionHandler@**: مدیریت استثنا.
- 18• **ControllerAdvice@**: مشاور سراسری کنترلرها.
- 19• **RestControllerAdvice@**: مشاور سراسری REST.
- 20• **ResponseStatus@**: تعیین کد وضعیت.

#### ۱۹.۴ Annotation های JPA

- 1 • **Entity@**: کلاس موجودیت.
- 2 • **Table@**: نام جدول.
- 3 • **Id@**: کلید اصلی.
- 4 • **GeneratedValue@**: تولید خودکار کلید.
- 5 • **Column@**: تنظیم ستون.
- 6 • **JoinColumn@**: ستون کلید خارجی.
- 7 • **OneToOne@**: رابطه یک به یک.
- 8 • **OneToMany@**: رابطه یک به چند.
- 9 • **ManyToOne@**: رابطه چند به یک.
- 10• **ManyToMany@**: رابطه چند به چند.
- 11• **Embedded@**: جاسازی کلاس.
- 12• **Embeddable@**: کلاس قابل جاسازی.
- 13• **MappedSuperclass@**: کلاس والد نگاشت شده.
- 14• **Inheritance@**: استراتژی وراثت.
- 15• **DiscriminatorColumn@**: ستون تشخیص نوع.

- 16• **DiscriminatorValue@**: مقدار تشخیص نوع.
- 17• **NamedQuery@**: کوئری نام‌دار.
- 18• **NamedQueries@**: چند کوئری نام‌دار.
- 19• **NamedEntityGraph@**: گراف موجودیت نام‌دار.
- 20• **NamedAttributeNode@**: گره attribute در گراف.
- 21• **Version@**: فیلد نسخه برای قفل خوش‌بینانه.
- 22• **Transient@**: فیلد غیرنگاشت‌شده.
- 23• **Temporal@**: تعیین نوع تاریخ.
- 24• **Enumerated@**: نگاشت **enum**.
- 25• **Lob@**: فیلد داده بزرگ.
- 26• **ElementCollection@**: مجموعه عناصر ساده.
- 27• **CollectionTable@**: جدول مجموعه.
- 28• **OrderColumn@**: ستون ترتیب.
- 29• **OrderBy@**: ترتیب پیش‌فرض.
- 30• **Where@**: شرط پیش‌فرض.
- 31• **Filter@**: فیلتر پویا.
- 32• **SQLDelete**: سفارشی حذف.
- 33• **SQLInsert**: سفارشی درج.
- 34• **SQLUpdate**: سفارشی به‌روزرسانی.

#### Spring Data JPA های Annotation ۱۹.۵

- 1• **Query@**: کوئری سفارشی.
- 2• **Param@**: نام‌گذاری پارامتر.
- 3• **Modifying@**: کوئری تغییردهنده.
- 4• **Transactional@**: مدیریت تراکنش.
- 5• **NoRepositoryBean@**: عدم ساخت Bean برای ریپازیتوری پایه.

#### Spring Security های Annotation ۱۹.۶

- 1• **PreAuthorize@**: بررسی مجوز قبل از متد.
- 2• **PostAuthorize@**: بررسی مجوز بعد از متد.
- 3• **PreFilter@**: فیلتر ورودی قبل از متد.
- 4• **PostFilter@**: فیلتر خروجی بعد از متد.
- 5• **Secured@**: بررسی نقش ساده.
- 6• **RolesAllowed@**: بررسی نقش (JSR-250).
- 7• **DenyAll@**: رد همه.
- 8• **PermitAll@**: اجازه به همه.
- 9• **AuthenticationPrincipal@**: تزریق کاربر جاری.

#### ۱۹.۷ Annotation های تست

- 1• **Test@**: متد تست.
- 2• **BeforeEach@**: قبل از هر تست.
- 3• **AfterEach@**: بعد از هر تست.
- 4• **BeforeAll@**: قبل از همه تست‌ها.
- 5• **AfterAll@**: بعد از همه تست‌ها.
- 6• **DisplayName@**: نام نمایشی.
- 7• **Disabled@**: غیرفعال‌سازی تست.
- 8• **ParameterizedTest@**: تست پارامتری.
- 9• **ValueSource@**: منبع مقادیر ساده.
- 10• **CsvSource@**: منبع CSV.
- 11• **EnumSource@**: منبع **enum**.
- 12• **MethodSource@**: منبع متد.
- 13• **Mock@**: ساخت mock.
- 14• **InjectMocks@**: تزریق mock‌ها.
- 15• **Spy@**: جاسوس.
- 16• **MockBean: Bean@**: جعلی Spring.
- 17• **SpyBean: Bean@**: جاسوس Spring.

- 18@ **SpringBootTest**: تست کامل Spring Boot.
- 19@ **WebMvcTest**: تست لایه وب.
- 20@ **DataJpaTest**: تست لایه JPA.
- 21@ **WebFluxTest**: تست WebFlux.
- 22@ **RestClientTest**: تست کلاینت REST.
- 23@ **JsonTest**: تست JSON.
- 24@ **TestConfiguration**: پیکربندی مخصوص تست.
- 25@ **ActiveProfiles**: فعال‌سازی profile.
- 26@ **Sql**: اجرای SQL قبل از تست.
- 27@ **Commit**: ثبت تراکنش تست.
- 28@ **Rollback**: بازگردانی تراکنش تست.
- 29@ **DirtyContext**: آلوده کردن context.

#### ۱۹.۸ Annotation های Lombok

- 1@ **Data**: تولید getter/setter/toString/equals/hashCode.
- 2@ **Getter**: تولید getter.
- 3@ **Setter**: تولید setter.
- 4@ **Builder**: تولید Builder.
- 5@ **NoArgsConstructor**: سازنده بدون آرگومان.
- 6@ **AllArgsConstructor**: سازنده همه آرگومان‌ها.
- 7@ **RequiredArgsConstructor**: سازنده فیلدهای **final**.
- 8@ **ToString**: تولید toString.
- 9@ **EqualsAndHashCode**: تولید equals/hashCode.
- 10@ **Slf4j**: تزریق Logger.
- 11@ **Value**: کلاس immutable.
- 12@ **NonNull**: بررسی **null**.
- 13@ **Cleanup**: بستن خودکار منبع.
- 14@ **SneakyThrows**: پرتاب پنهان استثنا.

## Jackson ۱۹.۹ Annotation های

- 1 @JsonProperty: نام JSON فیلد.
- 2 @JsonIgnore: نادیده گرفتن فیلد.
- 3 @JsonFormat: قالب بندی مقدار.
- 4 @JsonInclude: شرط درج فیلد.
- 5 @JsonCreator: سازنده deserialization.
- 6 @JsonValue: مقدار سریالایز.
- 7 @getter: JsonAnyGetter فیلدهای پویا.
- 8 @setter: JsonAnySetter فیلدهای پویا.
- 9 @JsonNaming: استراتژی نام گذاری.
- 10 @JsonSerialize: سریالایزر سفارشی.
- 11 @JsonDeserialize: دی سریالایزر سفارشی.

## Validation ۱۹.۱۰ Annotation های

- 1 @Valid: فعال سازی اعتبارسنجی.
- 2 @NotNull: نباید null باشد.
- 3 @NotEmpty: نباید خالی باشد.
- 4 @NotBlank: نباید فقط فضای خالی باشد.
- 5 @Size: محدودیت اندازه.
- 6 @Min: حداقل مقدار.
- 7 @Max: حداکثر مقدار.
- 8 @Email: قالب ایمیل.
- 9 @Pattern: الگوی regex.
- 10 @Positive: عدد مثبت.
- 11 @Negative: عدد منفی.
- 12 @Past: تاریخ گذشته.
- 13 @Future: تاریخ آینده.
- 14 @AssertTrue: باید true باشد.

15• AssertFalse@: باید false باشد.

## ♦ بخش ۲۰: کلمات کلیدی جاوا (Java Keywords)

### ۲۰.۱ کلمات کلیدی پایه

- 1 abstract: تعریف کلاس/متد انتزاعی.
- 2 assert: بررسی شرط در تست/دیباگ.
- 3 boolean: نوع داده منطقی.
- 4 break: خروج از حلقه/switch.
- 5 byte: نوع داده عددی ۸ بیتی.
- 6 case: شاخه switch.
- 7 catch: مدیریت استثنا.
- 8 char: نوع داده کاراکتر.
- 9 class: تعریف کلاس.
- 10 const: رزرو شده و استفاده نمی‌شود.
- 11 continue: پرش به تکرار بعدی حلقه.
- 12 default: شاخه پیش‌فرض switch / متد پیش‌فرض interface.
- 13 do: شروع حلقه do-while.
- 14 double: نوع داده اعشاری ۶۴ بیتی.
- 15 else: شاخه جایگزین if.
- 16 enum: تعریف نوع شمارشی.
- 17 extends: وراثت کلاس.
- 18 final: غیرقابل تغییر.
- 19 finally: بلوک همیشه اجرا.
- 20 float: نوع داده اعشاری ۳۲ بیتی.
- 21 for: حلقه شمارنده.
- 22 goto: رزرو شده و استفاده نمی‌شود.
- 23 if: شرط.

- 24• **implements**: پیاده‌سازی **interface**.
- 25• **import**: وارد کردن کلاس/پکیج.
- 26• **instanceof**: بررسی نوع.
- 27• **int**: نوع داده عددی ۳۲ بیتی.
- 28• **interface**: تعریف اینترفیس.
- 29• **long**: نوع داده عددی ۶۴ بیتی.
- 30• **native**: متد پیاده‌سازی شده به زبان بومی.
- 31• **new**: ساخت آبجکت.
- 32• **package**: تعریف پکیج.
- 33• **private**: دسترسی فقط داخل کلاس.
- 34• **protected**: دسترسی پکیج + زیرکلاس.
- 35• **public**: دسترسی عمومی.
- 36• **return**: بازگشت از متد.
- 37• **short**: نوع داده عددی ۱۶ بیتی.
- 38• **static**: متعلق به کلاس.
- 39• **strictfp**: محدودسازی محاسبات اعشاری به استاندارد.
- 40• **super**: ارجاع به کلاس والد.
- 41• **switch**: انتخاب چندشاخه‌ای.
- 42• **synchronized**: قفل هم‌زمانی.
- 43• **this**: ارجاع به آبجکت جاری.
- 44• **throw**: پرتاب استثنا.
- 45• **throws**: اعلام استثنا در امضای متد.
- 46• **transient**: فیلد غیرقابل سریالایز.
- 47• **try**: بلوک کد مشکوک به خطا.
- 48• **void**: بدون مقدار بازگشتی.
- 49• **volatile**: تضمین **visibility** متغیر بین **thread**ها.
- 50• **while**: حلقه شرطی.

- 1• **true**: مقدار منطقی درست.
- 2• **false**: مقدار منطقی نادرست.
- 3• **null**: مقدار پوچ reference.
- 4• **var**: استنتاج نوع متغیر محلی (جاوا +۱۰).
- 5• **record**: تعریف رکورد (جاوا +۱۶).
- 6• **sealed**: کلاس/اینترفیس محدود (جاوا +۱۷).
- 7• **permits**: تعیین زیرکلاس‌های مجاز.
- 8• **yield**: بازگشت مقدار از switch expression.
- 9• **non-sealed**: باز کردن کلاس sealed برای وراثت.

## ♦ بخش ۲۱: کلاس‌ها و اینترفیس‌های مهم جاوا

### ۲۱.۱ java.lang

- 1• **Object**: ریشه همه کلاس‌های جاوا.
- 2• **String**: رشته متن immutable.
- 3• **StringBuilder**: ساخت رشته mutable و سریع.
- 4• **StringBuffer**: ساخت رشته mutable و thread-safe.
- 5• **Math**: توابع ریاضی.
- 6• **System**: دسترسی به سیستم (in/out/exit).
- 7• **Runtime**: دسترسی به runtime JVM.
- 8• **Thread**: کلاس ریسه.
- 9• **Runnable**: اینترفیس task.
- 10• **Throwable**: ریشه خطاها و استثناها.
- 11• **Exception**: کلاس پایه استثناها.
- 12• **Error**: کلاس پایه خطاهای جدی.
- 13• **RuntimeException**: کلاس پایه استثناهای unchecked.
- 14• **Class**: نماینده کلاس در runtime.
- 15• **ClassLoader**: بارگذار کلاس‌ها.

- 16• **Integer**: wrapper کلاس **int**.
- 17• **Long**: wrapper کلاس **long**.
- 18• **Double**: wrapper کلاس **double**.
- 19• **Float**: wrapper کلاس **float**.
- 20• **Boolean**: wrapper کلاس **boolean**.
- 21• **Character**: wrapper کلاس **char**.
- 22• **Byte**: wrapper کلاس **byte**.
- 23• **Short**: wrapper کلاس **short**.
- 24• **Void**: نماینده نوع **void**.
- 25• **Process**: فرآیند سیستمی.
- 26• **ProcessBuilder**: سازنده فرآیند.
- 27• **SecurityManager**: مدیر امنیت (منسوخ).
- 28• **Package**: نماینده پکیج.
- 29• **Module**: نماینده ماژول (جاوا ۹+).

## ۲۱.۲ java.util

- 1• **List**: اینترفیس لیست.
- 2• **ArrayList**: پیاده‌سازی آرایه‌ای List.
- 3• **LinkedList**: پیاده‌سازی پیوندی List.
- 4• **Vector**: هم‌زمان (قدیمی).
- 5• **Stack**: پشته LIFO.
- 6• **Set**: اینترفیس مجموعه بدون تکرار.
- 7• **HashSet**: بر پایه hash.
- 8• **LinkedHashSet**: با حفظ ترتیب درج.
- 9• **TreeSet**: مرتب.
- 10• **Map**: اینترفیس کلید-مقدار.
- 11• **HashMap**: بر پایه hash.
- 12• **LinkedHashMap**: با حفظ ترتیب درج.

- 13• **TreeMap**: Map مرتب.
- 14• **Hashtable**: Map هم‌زمان (قدیمی).
- 15• **Queue**: اینترفیس صف.
- 16• **PriorityQueue**: صف اولویت‌دار.
- 17• **Deque**: صف دوطرفه.
- 18• **ArrayDeque**: Deque آرایه‌ای.
- 19• **Iterator**: پیمایشگر.
- 20• **ListIterator**: پیمایشگر لیست.
- 21• **Collections**: ابزارهای کالکشن.
- 22• **Arrays**: ابزارهای آرایه.
- 23• **Comparator**: مقایسه‌گر خارجی.
- 24• **Comparable**: مقایسه‌گر داخلی.
- 25• **Optional**: ظرف امن مقدار.
- 26• **Scanner**: خواندن ورودی.
- 27• **Random**: تولید اعداد تصادفی.
- 28• **UUID**: شناسه یکتا.
- 29• **Date**: تاریخ (قدیمی).
- 30• **Calendar**: تقویم (قدیمی).
- 31• **Timer**: زمان‌بند.
- 32• **TimerTask**: task زمان‌بند.
- 33• **Properties**: نگهداری کلید-مقدار تنظیمات.
- 34• **Locale**: نماینده locale.
- 35• **Currency**: نماینده ارز.
- 36• **Formatter**: قالب‌بندی خروجی.
- 37• **StringTokenizer**: تجزیه رشته (قدیمی).
- 38• **BitSet**: مجموعه بیتی.
- 39• **EnumMap**: Map برای enum.
- 40• **EnumSet**: Set برای enum.

- 41• **WeakHashMap**: Map با کلید weak.
- 42• **IdentityHashMap**: Map بر اساس ==.
- 43• **LinkedBlockingQueue**: صف مسدودکننده پیوندی.
- 44• **ArrayBlockingQueue**: صف مسدودکننده آرایه‌ای.
- 45• **PriorityBlockingQueue**: صف مسدودکننده اولویت‌دار.
- 46• **SynchronousQueue**: صف هم‌زمان.
- 47• **ConcurrentHashMap**: Map هم‌زمان امن.
- 48• **CopyOnWriteArrayList**: لیست امن با کپی.
- 49• **CopyOnWriteArraySet**: Set امن با کپی.
- 50• **AtomicInteger**: int اتمیک.
- 51• **AtomicLong**: long اتمیک.
- 52• **AtomicBoolean**: boolean اتمیک.
- 53• **AtomicReference**: reference اتمیک.
- 54• **CountDownLatch**: هماهنگ‌کننده انتظار.
- 55• **CyclicBarrier**: مانع چرخه‌ای.
- 56• **Semaphore**: سمافور.
- 57• **Exchanger**: تبادل داده بین threadها.
- 58• **Phaser**: هماهنگ‌کننده چندفازی.

### ۲۱.۳ java.util.function

- 1 • **<Predicate>T**: ورودی **boolean**  $T \rightarrow$ .
- 2 • **<Function>T,R**: ورودی  $T \rightarrow$  خروجی  $R$ .
- 3 • **<Consumer>T**: ورودی  $T \rightarrow$  بدون خروجی.
- 4 • **<Supplier>T**: بدون ورودی  $\rightarrow$  خروجی  $T$ .
- 5 • **<BiFunction>T,U,R**: دو ورودی  $\rightarrow$  خروجی  $R$ .
- 6 • **<BiPredicate>T,U**: دو ورودی  $\rightarrow$  **boolean**.
- 7 • **<BiConsumer>T,U**: دو ورودی  $\rightarrow$  بدون خروجی.
- 8 • **<UnaryOperator>T**: ورودی  $T \rightarrow$  خروجی  $T$ .

- 9 `BinaryOperator<T>`: دو `T → T`.
- 10 `IntFunction<R>`: `int → R`.
- 11 `IntPredicate`: `int → boolean`.
- 12 `IntConsumer`: بدون خروجی. `int →`.
- 13 `IntSupplier`: بدون ورودی `int →`.
- 14 `LongFunction<R>`: `long → R`.
- 15 `DoubleFunction<R>`: `double → R`.
- 16 `ToIntFunction<T>`: `T → int`.
- 17 `ToLongFunction<T>`: `T → long`.
- 18 `ToDoubleFunction<T>`: `T → double`.

#### ۲۱.۴ java.util.stream

- 1 `Stream`: جریان داده.
- 2 `IntStream`: `Stream int`.
- 3 `LongStream`: `Stream long`.
- 4 `DoubleStream`: `Stream double`.
- 5 `Collectors`: جمع‌آورنده‌های `Stream`.
- 6 `StreamSupport`: ابزار ساخت `Stream`.
- 7 `BaseStream`: اینترفیس پایه `Stream`.

#### ۲۱.۵ java.util.concurrent

- 1 `Executor`: اینترفیس اجرای `task`.
- 2 `ExecutorService`: مدیریت `thread pool`.
- 3 `Executors`: کارخانه ساخت `executor`.
- 4 `ThreadPoolExecutor`: پیاده‌سازی `thread pool`.
- 5 `ScheduledExecutorService`: `executor` زمان‌بندی‌شده.
- 6 `ScheduledThreadPoolExecutor`: پیاده‌سازی آن.
- 7 `ForkJoinPool`: استخر `Fork/Join`.

- **Future 8**: نتیجه آینده.
- **CompletableFuture: Future 9**: کامل تر و `async`.
- **Callable: task 10**: با خروجی.
- **TimeUnit 11**: واحد زمان.
- **Lock 12**: اینترفیس قفل.
- **ReentrantLock 13**: قفل قابل ورود مجدد.
- **ReentrantReadWriteLock 14**: قفل خواندن/نوشتن.
- **ReadWriteLock 15**: اینترفیس قفل خواندن/نوشتن.
- **Condition 16**: شرط انتظار.
- **StampedLock 17**: قفل نسخه دار.
- **AbstractQueuedSynchronizer 18**: پایه ساخت `synchronizer`.
- **ConcurrentLinkedQueue 19**: صف پیوندی همزمان.
- **ConcurrentLinkedDeque: Deque 20**: پیوندی همزمان.
- **ConcurrentSkipListMap: Map 21**: همزمان مبتنی بر `skip list`.
- **ConcurrentSkipListSet: Set 22**: همزمان مبتنی بر `skip list`.
- **ThreadLocalRandom: Random 23**: مخصوص `thread`.

۲۱.۶ [java.io](http://java.io)

- **File 1**: نماینده فایل/دایرکتوری.
- **InputStream 2**: پایه `stream` ورودی بایت.
- **OutputStream 3**: پایه `stream` خروجی بایت.
- **Reader 4**: پایه `stream` ورودی کاراکتر.
- **Writer 5**: پایه `stream` خروجی کاراکتر.
- **FileInputStream 6**: خواندن بایت از فایل.
- **FileOutputStream 7**: نوشتن بایت در فایل.
- **FileReader 8**: خواندن کاراکتر از فایل.
- **FileWriter 9**: نوشتن کاراکتر در فایل.
- **BufferedInputStream 10**: بافر ورودی بایت.

- 11• **BufferedOutputStream**: بافر خروجی بایت.
- 12• **BufferedReader**: بافر ورودی کاراکتر.
- 13• **BufferedWriter**: بافر خروجی کاراکتر.
- 14• **DataInputStream**: خواندن primitive.
- 15• **DataOutputStream**: نوشتن primitive.
- 16• **ObjectInputStream**: خواندن آبجکت.
- 17• **ObjectOutputStream**: نوشتن آبجکت.
- 18• **PrintStream**: چاپ داده.
- 19• **PrintWriter**: چاپ متن.
- 20• **PipedInputStream: stream** ورودی لوله‌ای.
- 21• **PipedOutputStream: stream** خروجی لوله‌ای.
- 22• **SequenceInputStream**: ادغام stream‌های ورودی.
- 23• **LineNumberReader**: خواندن خط با شماره.
- 24• **PushbackReader**: امکان برگرداندن کاراکتر.
- 25• **RandomAccessFile**: دسترسی تصادفی به فایل.
- 26• **Serializable**: اینترفیس سریالایز.
- 27• **Externalizable**: سریالایز سفارشی.
- 28• **IOException**: خطای I/O.
- 29• **FileNotFoundException**: خطای پیدا نشدن فایل.
- 30• **EOFException**: خطای پایان فایل.
- 31• **InterruptedIOException**: خطای وقفه I/O.
- 32• **UncheckedIOException**: خطای unchecked I/O.

## ۲۱.۷ java.nio

- 1 • **Buffer**: ظرف داده NIO.
- 2 • **ByteBuffer: Buffer** بایت.
- 3 • **CharBuffer: Buffer** کاراکتر.
- 4 • **ShortBuffer: Buffer** short.

- **IntBuffer**: Buffer **int** 5
- **LongBuffer**: Buffer **long** 6
- **FloatBuffer**: Buffer **float** 7
- **DoubleBuffer**: Buffer **double** 8
- **Channel**: کانال I/O 9
- **FileChannel**: کانال فایل 10
- **SocketChannel**: کانال سوکت 11
- **ServerSocketChannel**: کانال سرور سوکت 12
- **DatagramChannel**: کانال UDP 13
- **Selector**: انتخابگر کانال‌ها 14
- **SelectionKey**: کلید انتخاب 15
- **Paths**: ابزار ساخت Path 16
- **Path**: نماینده مسیر 17
- **Files**: ابزار کار با فایل 18
- **FileSystems**: ابزار ساخت FileSystem 19
- **FileSystem**: نماینده فایل سیستم 20
- **FileStore**: نماینده انبار فایل 21
- **WatchService**: سرویس پایش تغییرات 22
- **WatchKey**: کلید پایش 23
- **StandardOpenOption**: گزینه‌های باز کردن فایل 24
- **StandardCopyOption**: گزینه‌های کپی 25
- **StandardCharsets**: Charsets: های استاندارد 26
- **Charset**: کدگذاری کاراکتر 27
- **CharsetDecoder**: رمزگشای کاراکتر 28
- **CharsetEncoder**: رمزگذار کاراکتر 29

۲۱.۸ [java.net](http://java.net)

- **Socket** 1: سوکت کلاینت

- **2 ServerSocket**: سوکت سرور.
- **3 DatagramSocket**: سوکت UDP.
- **4 DatagramPacket**: بسته UDP.
- **5 InetAddress**: آدرس IP.
- **6 URL**: آدرس منبع.
- **7 URI**: شناسه منبع.
- **8 URLConnection**: اتصال URL.
- **9 HttpURLConnection**: اتصال HTTP.
- **10 InetSocketAddress**: آدرس سوکت.
- **11 Proxy**: پروکسی.
- **12 NetworkInterface**: واسط شبکه.
- **13 Cookie**: کوکی HTTP.
- **14 CookieManager**: مدیر کوکی.
- **15 Authenticator**: احراز هویت کننده.
- **16 PasswordAuthentication**: احراز هویت نام/رمز.

## ۲۱.۹ java.sql

- **1 Driver**: درایور دیتابیس.
- **2 DriverManager**: مدیر درایورها.
- **3 Connection**: اتصال دیتابیس.
- **4 Statement**: اجرای کوئری.
- **5 PreparedStatement**: کوئری پارامتری.
- **6 CallableStatement**: فراخوانی stored procedure.
- **7 ResultSet**: نتیجه کوئری.
- **8 ResultSetMetaData**: متادیتای نتیجه.
- **9 DatabaseMetaData**: متادیتای دیتابیس.
- **10 SQLException**: خطای SQL.
- **11 BatchUpdateException**: خطای batch.

- 12• **SQLWarning**: هشدار SQL.
- 13• **Savepoint**: نقطه ذخیره تراکنش.
- 14• **Types**: ثابت‌های نوع SQL.
- 15• **Date**: تاریخ SQL.
- 16• **Time**: زمان SQL.
- 17• **Timestamp**: زمان SQL estamp.
- 18• **Blob**: داده باینری بزرگ.
- 19• **Clob**: داده متنی بزرگ.
- 20• **Array**: آرایه SQL.
- 21• **Ref**: ارجاع SQL.
- 22• **RowId**: شناسه ردیف.
- 23• **SQLData**: نگاشت نوع سفارشی.
- 24• **SQLInput: stream**: ورودی SQL.
- 25• **SQLOutput: stream**: خروجی SQL.

#### ۲۱.۱۰ java.math

- 1• **BigInteger**: عدد صحیح دلخواه بزرگ.
- 2• **BigDecimal**: عدد اعشاری دقیق.
- 3• **MathContext**: تنظیم دقت.
- 4• **RoundingMode**: حالت گرد کردن.

#### ۲۱.۱۱ java.text

- 1• **DateFormat**: قالب‌بندی تاریخ.
- 2• **SimpleDateFormat**: پیاده‌سازی DateFormat.
- 3• **NumberFormat**: قالب‌بندی عدد.
- 4• **DecimalFormat**: پیاده‌سازی NumberFormat.
- 5• **MessageFormat**: قالب‌بندی پیام.
- 6• **ChoiceFormat**: قالب‌بندی شرطی.

- **Collator 7**: مقایسه رشته‌ها.
- **RuleBasedCollator: Collator 8**: قاعده‌محور.
- **DateFormatSymbols 9**: نمادهای تاریخ.
- **DecimalFormatSymbols 10**: نمادهای عدد.
- **ParsePosition 11**: موقعیت تجزیه.
- **FieldPosition 12**: موقعیت فیلد.
- **Format 13**: پایه قالب‌بندی.
- **AttributedCharacterIterator 14**: پیمایشگر کاراکتر با attribute.
- **Normalizer 15**: نرمال‌سازی یونیکد.

## ۲۱.۱۲ java.time

- **LocalDate 1**: تاریخ بدون زمان.
- **LocalTime 2**: زمان بدون تاریخ.
- **LocalDateTime 3**: تاریخ و زمان.
- **ZonedDateTime 4**: تاریخ/زمان با منطقه.
- **OffsetDateTime 5**: تاریخ/زمان با offset.
- **OffsetTime 6**: زمان با offset.
- **Instant 7**: لحظه دقیق UTC.
- **Duration 8**: مدت زمان.
- **Period 9**: دوره تاریخ.
- **ZoneId 10**: شناسه منطقه زمانی.
- **ZoneOffset: offset 11**: منطقه.
- **Clock 12**: ساعت.
- **DayOfWeek 13**: روز هفته.
- **Month 14**: ماه.
- **Year 15**: سال.
- **YearMonth 16**: سال-ماه.
- **MonthDay 17**: ماه-روز.

- 18• **DateTimeFormatter**: قالب‌بندی تاریخ/زمان.
- 19• **DateTimeFormatterBuilder**: سازنده formatter.
- 20• **Temporal**: اینترفیس موجودیت زمانی.
- 21• **TemporalAccessor**: دسترسی به فیلدهای زمانی.
- 22• **TemporalAdjuster**: تنظیم‌کننده زمانی.
- 23• **TemporalAdjusters**: تنظیم‌کننده‌های آماده.
- 24• **TemporalAmount**: مقدار زمانی.
- 25• **TemporalField**: فیلد زمانی.
- 26• **TemporalUnit**: واحد زمانی.
- 27• **ChronoField**: فیلد استاندارد.
- 28• **ChronoUnit**: واحد استاندارد.
- 29• **ChronoLocalDate**: تاریخ گاه‌شماری.
- 30• **ChronoLocalDateTime**: تاریخ/زمان گاه‌شماری.
- 31• **ChronoZonedDateTime**: تاریخ/زمان منطقه‌ای گاه‌شماری.
- 32• **Chronology**: گاه‌شماری.
- 33• **IsoChronology**: گاه‌شماری ISO.
- 34• **HijrahChronology**: گاه‌شماری هجری.
- 35• **JapaneseChronology**: گاه‌شماری ژاپنی.
- 36• **MinguoChronology**: گاه‌شماری مینگو.
- 37• **ThaiBuddhistChronology**: گاه‌شماری بودایی تایلندی.

#### ۲۱.۱۳ java.lang.reflect

- 1• **Class**: نماینده کلاس.
- 2• **Field**: فیلد بازتابی.
- 3• **Method**: متد بازتابی.
- 4• **Constructor**: سازنده بازتابی.
- 5• **Modifier**: تغییردهنده‌ها.
- 6• **Array**: ابزار آرایه بازتابی.

- **7 Proxy**: پروکسی پویا.
- **8 InvocationHandler**: هندلر فراخوانی پروکسی.
- **9 MethodHandles**: ابزار method handle.
- **10 MethodHandle**: ارجاع اجرایی متد.
- **11 MethodType**: نوع متد.
- **12 VarHandle**: ارجاع به متغیر.
- **13 Parameter**: پارامتر متد.
- **14 AnnotatedElement**: عنصر دارای انوتیشن.
- **15 Annotation**: انوتیشن.
- **16 AccessibleObject**: پایه فیلد/متد/سازنده.
- **17 ReflectPermission**: مجوز بازتاب.
- **18 Type**: نوع جنریک.
- **19 ParameterizedType**: نوع پارامتری.
- **20 GenericArrayType**: آرایه جنریک.
- **21 WildcardType**: نوع wildcard.
- **22 TypeVariable**: متغیر نوع.

#### ۲۱.۱۴ java.lang.annotation

- **1 Annotation**: اینترفیس پایه انوتیشن.
- **2 Retention**: سیاست نگهداری.
- **3 RetentionPolicy**: مقادیر سیاست نگهداری.
- **4 Target**: محل مجاز.
- **5 ElementType**: انواع عناصر هدف.
- **6 Documented**: درج در مستندات.
- **7 Inherited**: ارث‌بری.
- **8 Repeatable**: تکرارپذیری.
- **9 Native**: نشانه بومی.
- **10 AnnotationFormatError**: خطای قالب انوتیشن.

11• **AnnotationTypeMismatchException**: عدم تطابق نوع.

12• **IncompleteAnnotationException**: انوتیشن ناقص.

#### ۲۱.۱۵ java.util.regex

1• **Pattern**: الگوی regex.

2• **Matcher**: تطبیق دهنده.

3• **MatchResult**: نتیجه تطبیق.

4• **PatternSyntaxException**: خطای نحو regex.

#### ۲۱.۱۶ java.util.zip

1• **ZipFile**: فایل ZIP.

2• **ZipEntry**: ورودی ZIP.

3• **ZipInputStream: stream**: ورودی ZIP.

4• **ZipOutputStream: stream**: خروجی ZIP.

5• **GZIPInputStream: stream**: ورودی GZIP.

6• **GZIPOutputStream: stream**: خروجی GZIP.

7• **Inflater**: بازکننده فشرده سازی.

8• **Deflater**: فشرده ساز.

9• **InflaterInputStream: stream**: بازکننده.

10• **DeflaterOutputStream: stream**: فشرده ساز.

11• **CRC32**: چک سام CRC32.

12• **Adler32**: چک سام Adler32.

13• **CheckedInputStream: stream**: با چک سام.

14• **CheckedOutputStream: stream**: با چک سام.

#### ۲۱.۱۷ java.util.logging

1• **Logger**: لاگر.

2• **Level**: سطح لاگ.

- **3 Handler**: هندلر لاگ.
- **4 ConsoleHandler**: هندلر کنسول.
- **5 FileHandler**: هندلر فایل.
- **6 SocketHandler**: هندلر سوکت.
- **7 StreamHandler**: هندلر stream.
- **8 MemoryHandler**: هندلر حافظه.
- **9 Formatter**: قالب بندی لاگ.
- **10 SimpleFormatter**: قالب ساده.
- **11 XMLFormatter**: قالب XML.
- **12 LogRecord**: رکورد لاگ.
- **13 LogManager**: مدیر لاگ.
- **14 Filter**: فیلتر لاگ.
- **15 ErrorManager**: مدیر خطا.

## ◆ ادامه بخش ۲۱: کلاس ها و اینترفیس های مهم جاوا (فقط مفاهیم)

### Swing / AWT ۲۱.۱۸

- **1 Swing**: کتابخانه UI قدیمی جاوا برای ساخت پنجره و دکمه.
- **2 AWT**: کتابخانه گرافیکی قدیمی تر و وابسته به سیستم عامل.
- **3 Layout Manager**: مدیر چیدمان اجزای UI.
- **4 Event Listener**: شنونده رویدادهای UI.
- **5 Adapter Class**: کلاس میانجی برای پیاده سازی ساده listener.

### Applet ۲۱.۱۹ (منسوخ)

- **1 Applet**: برنامه کوچک جاوا که در مرورگر اجرا می شود (منسوخ).
- **2 AppletContext**: محیط اجرای Applet.
- **3 AudioClip**: پخش صدا در Applet.

### Servlet ۲۱.۲۰

- **Servlet 1**: کامپوننت سمت سرور برای پردازش درخواست HTTP.
- **Servlet 2**: **GenericServlet** مستقل از پروتکل.
- **Servlet 3**: **HttpServlet** مخصوص HTTP.
- **Servlet 4**: **ServletRequest / HttpServletRequest**: نماینده درخواست کلاینت.
- **Servlet 5**: **ServletResponse / HttpServletResponse**: نماینده پاسخ سرور.
- **Servlet 6**: **ServletConfig**: تنظیمات اختصاصی یک Servlet.
- **Servlet 7**: **ServletContext**: بستر مشترک کل وب اپلیکیشن.
- **Servlet 8**: **RequestDispatcher**: ارسال درخواست به منبع دیگر.
- **Servlet 9**: **Filter**: فیلتر پردازش درخواست/پاسخ.
- **Servlet 10**: **FilterChain**: زنجیره فیلترها.
- **Servlet 11**: **HttpSession**: نشست کاربر.
- **Servlet 12**: **Cookie**: داده ذخیره شده در مرورگر.
- **Servlet 13**: **AsyncContext**: پردازش غیرهمزمان درخواست.
- **Servlet 14**: **DispatcherType**: نوع (REQUEST/FORWARD/INCLUDE) **dispatch**.
- **Servlet 15**: **MultipartConfig**: پیکربندی آپلود فایل.
- **Servlet 16**: **Listener**: شنونده رویدادهای چرخه حیات Servlet.

## JSP ۲۱.۲۱

- **JSP 1**: فناوری قالب ساز سمت **سرور** (منسوخ تر از Thymeleaf).
- **JSP 2**: **JspWriter**: نویسنده خروجی JSP.
- **JSP 3**: **PageContext: context**: صفحه JSP.
- **JSP 4**: **Tag**: تگ سفارشی JSP.
- **JSP 5**: **TagSupport**: پایه ساخت تگ سفارشی.
- **JSP 6**: **SimpleTag**: تگ ساده JSP.
- **JSP 7**: **TagExtraInfo**: متادیتای تگ.
- **JSP 8**: **TagLibraryInfo**: اطلاعات کتابخانه تگ.
- **JSP 9**: **JspFragment**: قطعه قابل استفاده مجدد JSP.
- **JSP 10**: **DynamicAttributes**: پشتیبانی attribute پویا.

- 1 **EntityManagerFactory**: کارخانه ساخت EntityManager.
- 2 **EntityManager**: مدیر موجودیت‌ها و تراکنش‌ها.
- 3 **EntityTransaction**: مدیریت تراکنش.
- 4 **Persistence**: ابزار ساخت EntityManagerFactory.
- 5 **Query / TypedQuery**: اجرای کوئری JPQL.
- 6 **StoredProcedureQuery**: فراخوانی stored procedure.
- 7 **Parameter**: پارامتر کوئری.
- 8 **Tuple / TupleElement**: نتیجه چندستونی.
- 9 **EntityGraph**: گراف بارگذاری موجودیت.
- 10 **Metamodel**: مدل متادیتای موجودیت‌ها.
- 11 **EntityType / EmbeddableType / MappedSuperclassType**: انواع موجودیت.
- 12 **ManagedType**: نوع مدیریت‌شده JPA.
- 13 **SingularAttribute / PluralAttribute: attribute**: تک/چندمقداری.
- 14 **Join / Fetch**: اتصال و بارگذاری رابطه.
- 15 **FetchType**: نحوه بارگذاری (EAGER/LAZY).
- 16 **CascadeType**: نوع انتشار عملیات.
- 17 **LockModeType**: حالت قفل.
- 18 **FlushModeType**: حالت flush.
- 19 **GenerationType**: استراتژی تولید کلید.
- 20 **TemporalType**: نوع تاریخ.
- 21 **EnumType**: نحوه ذخیره enum.
- 22 **PersistenceContextType**: نوع context.
- 23 **AccessType**: نوع دسترسی (FIELD/PROPERTY).
- 24 **CallbackType**: نوع callback چرخه حیات.
- 25 **SchemaGeneration**: تولید schema.
- 26 **Index / UniqueConstraint / ForeignKey**: قیدهای دیتابیس.
- 27 **NamedQuery / NamedNativeQuery**: کوئری نام‌دار.

- 28• **SqlResultSetMapping**: نگاشت نتیجه SQL.
- 29• **EntityListeners**: شنونده‌های چرخه حیات موجودیت.
- 30• **PrePersist / PostPersist / PreUpdate / PostUpdate / PreRemove / PostRemove / PostLoad: callback** های چرخه حیات.

## ♦ بخش ۲۲: ابزارها و تکنولوژی‌های جانبی

### ۲۲.۱ ابزارهای Build

- 1• **Maven**: ابزار build و مدیریت وابستگی مبتنی بر XML.
- 2• **Gradle**: ابزار build مدرن مبتنی بر DSL.
- 3• **Ant**: ابزار build قدیمی و دستی.
- 4• **Bazel**: ابزار build مقیاس‌پذیر گوگل.
- 5• **SBT**: ابزار build مخصوص Scala.

### ۲۲.۲ سرورهای اپلیکیشن

- 1• **Tomcat**: کانتنر Servlet سبک.
- 2• **Jetty**: کانتنر Servlet سبک و embeddable.
- 3• **Undertow**: وب‌سرور سبک JBoss.
- 4• **WildFly**: سرور اپلیکیشن JBoss.
- 5• **WebLogic**: سرور اپلیکیشن Oracle.
- 6• **WebSphere**: سرور اپلیکیشن IBM.
- 7• **GlassFish**: سرور اپلیکیشن مرجع Jakarta EE.
- 8• **JBoss**: سرور اپلیکیشن قدیمی RedHat.
- 9• **Netty**: فریمورک شبکه غیرهم‌زمان.

### ۲۲.۳ دیتابیس‌ها

- 1• **PostgreSQL**: دیتابیس رابطه‌ای پیشرفته متن‌باز.
- 2• **MySQL**: دیتابیس رابطه‌ای محبوب.

- **MariaDB 3**: انشعاب متن باز MySQL.
- **Oracle 4**: دیتابیس تجاری Oracle.
- **SQL Server 5**: دیتابیس مایکروسافت.
- **DB2 6**: دیتابیس IBM.
- **SQLite 7**: دیتابیس سبک فایلی.
- **H2 8**: دیتابیس درون حافظه ای جاوا.
- **HSQLDB 9**: دیتابیس درون حافظه ای جاوا.
- **Derby 10**: دیتابیس درون حافظه ای جاوا.
- **MongoDB 11**: دیتابیس NoSQL سندی.
- **Cassandra 12**: دیتابیس توزیع شده ستونی.
- **Redis 13**: دیتابیس/کش کلید-مقدار درون حافظه ای.
- **Neo4j 14**: دیتابیس گرافی.
- **Elasticsearch 15**: موتور جستجو و تحلیل.
- **InfluxDB 16**: دیتابیس سری زمانی.

#### ۲۲.۴ ابزارهای تست

- **JUnit 1**: فریمورک تست اصلی جاوا.
- **TestNG 2**: فریمورک تست جایگزین JUnit.
- **Mockito 3**: فریمورک mock سازی.
- **EasyMock 4**: فریمورک mock قدیمی.
- **PowerMock: mock 5**: کردن **static/final**.
- **AssertJ: assertion 6**: های روان.
- **Hamcrest: matcher 7**: های تست.
- **Selenium 8**: تست خودکار مرورگر.
- **Cucumber 9**: تست BDD.
- **REST Assured 10**: تست API های REST.
- **WireMock: mock 11**: سرور HTTP.
- **Testcontainers 12**: اجرای Docker در تست.

13 • **JaCoCo**: سنجش پوشش کد.

14 • **Cobertura**: سنجش پوشش کد قدیمی.

15 • **SonarQube**: تحلیل کیفیت کد.

#### ۲۲.۵ ابزارهای CI/CD

1 • **Jenkins**: سرور CI/CD متن باز.

2 • **GitHub Actions**: CI/CD داخلی GitHub.

3 • **GitLab CI**: CI/CD داخلی GitLab.

4 • **CircleCI**: CI/CD ابری.

5 • **Travis CI**: CI/CD ابری قدیمی.

6 • **TeamCity**: CI/CD JetBrains.

7 • **Bamboo**: CI/CD Atlassian.

8 • **Azure DevOps**: CI/CD مایکروسافت.

9 • **ArgoCD**: استقرار GitOps در Kubernetes.

10 • **Spinnaker**: استقرار مداوم چندابری.

#### ۲۲.۶ ابزارهای Container

1 • **Docker**: پلتفرم کانتینری سازی.

2 • **Podman**: جایگزین بدون daemon برای Docker.

3 • **containerd**: runtime کانتینر.

4 • **CRI-O**: runtime کانتینر Kubernetes.

5 • **Kubernetes**: ارکستراتور کانتینر.

6 • **OpenShift**: پلتفرم Kubernetes ردهت.

7 • **Docker Swarm**: ارکستراتور بومی Docker.

8 • **Nomad**: ارکستراتور HashiCorp.

9 • **Rancher**: مدیریت Kubernetes.

#### ۲۲.۷ ابزارهای Monitoring

- **Prometheus 1**: سیستم مانیتورینگ و هشدار.
- **Grafana 2**: داشبورد بصری مانیتورینگ.
- **ELK Stack: Elasticsearch + Logstash + Kibana 3**.
- **Elasticsearch 4**: ذخیره و جستجوی لاگ.
- **Logstash 5**: جمع‌آوری و پردازش لاگ.
- **Kibana 6**: نمایش لاگ.
- **Zipkin 7**: ردیابی توزیع‌شده.
- **Jaeger 8**: ردیابی توزیع‌شده Uber.
- **Sleuth 9**: ردیابی Spring Cloud.
- **Micrometer 10**: متریک‌های اپلیکیشن.
- **New Relic 11**: مانیتورینگ APM.
- **Datadog 12**: مانیتورینگ ابری.
- **AppDynamics 13**: مانیتورینگ APM.
- **Dynatrace 14**: مانیتورینگ APM.

#### ۲۲.۸ ابزارهای Message Broker

- **Apache Kafka 1**: پلتفرم streaming توزیع‌شده.
- **RabbitMQ: broker 2**: پیام AMQP.
- **ActiveMQ: broker 3**: پیام جاوا.
- **Redis Pub/Sub 4**: پیام‌رسانی سبک Redis.
- **NATS 5**: پیام‌رسانی سبک و سریع.
- **Pulsar 6**: پیام‌رسانی توزیع‌شده.

#### ۲۲.۹ ابزارهای Caching

- **Redis 1**: کش درون‌حافظه‌ای.
- **Ehcache 2**: کش جاوا.
- **Caffeine 3**: کش سریع جاوا.
- **Hazelcast 4**: کش توزیع‌شده.

5• **Infinispan**: کش توزیع شده JBoss.

6• **Memcached**: کش توزیع شده ساده.

#### ۲۲.۱۰ ابزارهای API

1• **Swagger**: مستندسازی API.

2• **OpenAPI**: استاندارد توصیف API.

3• **Postman**: تست API.

4• **Insomnia**: تست API.

5• **SoapUI**: تست سرویس‌های SOAP/REST.

6• **GraphQL**: زبان کوئری API.

#### ۲۲.۱۱ Version Control ابزارهای

1• **Git**: سیستم کنترل نسخه توزیع شده.

2• **SVN**: سیستم کنترل نسخه متمرکز.

3• **Mercurial**: سیستم کنترل نسخه توزیع شده.

4• **Bitbucket**: میزبانی Git Atlassian.

5• **GitHub**: میزبانی Git مایکروسافت.

6• **GitLab**: میزبانی Git متن باز.

#### ۲۲.۱۲ IDE ابزارهای

1• **IntelliJ IDEA**: محبوب JetBrains.

2• **Eclipse**: متن باز.

3• **NetBeans**: متن باز Oracle.

4• **VS Code**: ویرایشگر سبک مایکروسافت.

5• **JDeveloper**: IDE Oracle.

6• **BlueJ**: آموزشی.

#### ۲۲.۱۳ Static Analysis ابزارهای

1• **Checkstyle**: بررسی سبک کد.

2• **PMD**: تحلیل کد.

3• **SpotBugs**: یاب باگ.

4• **FindBugs**: نسخه قدیمی SpotBugs.

5• **SonarLint**: تحلیل کیفیت کد.

6• **Error Prone**: بررسی خطاهای رایج.

#### ۲۲.۱۴ ابزارهای Profiling

1• **VisualVM**: پروفایلر بصری JVM.

2• **JProfiler**: پروفایلر تجاری.

3• **YourKit**: پروفایلر تجاری.

4• **Java Mission Control**: پروفایلر Oracle.

5• **JConsole**: مانیتور JVM.

6• **MAT** (Memory Analyzer Tool): تحلیل heap dump.

7• **Async Profiler**: پروفایلر کم هزینه.

#### ۲۲.۱۵ Dependency Management

1• **Maven**: مدیریت وابستگی.

2• **Gradle**: مدیریت وابستگی.

3• **Ivy**: مدیریت وابستگی Ant.

4• **Nexus**: مخزن وابستگی.

5• **Artifactory**: مخزن وابستگی JFrog.

#### ۲۲.۱۶ Configuration Management

1• **Ansible**: مدیریت پیکربندی agentless.

2• **Puppet**: مدیریت پیکربندی.

3• **Chef**: مدیریت پیکربندی.

4• **SaltStack**: مدیریت پیکربندی.

5• **Terraform**: زیرساخت به صورت کد.

#### ۲۲.۱۷ ابزارهای Virtualization

1• **VMware**: مجازی سازی تجاری.

2• **VirtualBox**: مجازی سازی متن باز Oracle.

3• **Hyper-V**: مجازی سازی مایکروسافت.

4• **KVM**: مجازی سازی متن باز Linux.

5• **Xen**: هایپروایزر متن باز.

#### ۲۲.۱۸ ابزارهای Cloud

1• **AWS**: ابر آمازون.

2• **Azure**: ابر مایکروسافت.

3• **Google Cloud**: ابر گوگل.

4• **Heroku**: PaaS ساده.

5• **DigitalOcean**: ابر ساده و ارزان.

6• **Linode**: ابر ساده.

7• **Oracle Cloud**: ابر Oracle.

8• **IBM Cloud**: ابر IBM.

### ♦ بخش ۲۳: مفاهیم پیشرفته Spring

#### ۲۳.۱ Spring Core

1• **IoC Container**: ظرف مدیریت چرخه حیات و وابستگی Bean ها.

2• **ApplicationContext**: context اصلی Spring با امکانات کامل تر.

3• **BeanFactory**: ظرف پایه و سبک Spring.

4• **BeanDefinition**: تعریف متادیتای یک Bean.

5• **BeanPostProcessor**: پردازشگر قبل/بعد از مقداردهی Bean.

6• **BeanFactoryPostProcessor**: پردازشگر متادیتای BeanFactory.

- **7 ApplicationListener**: شنونده رویدادهای اپلیکیشن.
- **8 ApplicationEvent**: رویداد اپلیکیشن.
- **9 MessageSource**: منبع پیام برای i18n.
- **10 ResourceLoader**: بارگذار منابع.
- **11 Environment**: محیط اجرا (propertyها و profileها).
- **12 PropertyResolver**: حل‌کننده property.
- **13 Profiles**: پروفایل‌های محیطی.
- **14 SpEL**: زبان عبارت Spring برای دسترسی پویا به مقادیر.

## Spring AOP ۲۳.۲

- **1 Aspect**: جنبه؛ ماژول‌سازی concern عرضی.
- **2 JoinPoint**: نقطه قابل اتصال در اجرای برنامه.
- **3 Pointcut**: تعریف شرط انتخاب JoinPoint.
- **4 Advice**: عملیات اجراشده در JoinPoint.
- **5 Before**: قبل از متد.
- **6 After**: بعد از متد.
- **7 AfterReturning**: بعد از بازگشت موفق.
- **8 AfterThrowing**: بعد از پرتاب استثنا.
- **9 Around**: دور متد (قبل و بعد).
- **10 Introduction**: افزودن متد/فیلد جدید به کلاس.
- **11 Weaving**: اتصال aspect به کد هدف.
- **12 Proxy**: واسط ایجادشده برای AOP.
- **13 Aspect@**: تعریف Aspect.
- **14 Pointcut@**: تعریف Pointcut.
- **15 @Before / @After / @AfterReturning / @AfterThrowing / @Around@**: انواع Advice.
- **16 EnableAspectJAutoProxy@**: فعال‌سازی AOP خودکار.

## Spring MVC ۲۳.۳

- 1 **DispatcherServlet**: کنترلر جلویی و مرکز MVC.
- 2 **HandlerMapping**: نگاشت درخواست به handler.
- 3 **HandlerAdapter**: اجرای handler.
- 4 **ViewResolver**: تشخیص view از نام.
- 5 **View**: نمای خروجی.
- 6 **ModelAndView**: مدل + view.
- 7 **Model / ModelMap**: ظرف داده مدل.
- 8 **Controller / RestController**: کنترلر MVC/REST.
- 9 **@RequestMapping** و مشتقاتش: نگاشت مسیر.
- 10 **@RequestParam / PathVariable**: استخراج پارامتر.
- 11 **@ResponseBody /RequestBody**: بدنه درخواست/پاسخ.
- 12 **@ModelAttribute**: اتصال داده به مدل.
- 13 **@CookieValue / @RequestHeader**: هدر و کوکی.
- 14 **@RequestAttribute / @SessionAttribute**: نشست/درخواست.
- 15 **@CrossOrigin**: فعال سازی CORS.
- 16 **@ExceptionHandler**: مدیریت استثنا.
- 17 **@RestControllerAdvice / @ControllerAdvice**: مشاور سراسری.
- 18 **@ResponseStatus**: تعیین کد وضعیت.
- 19 **ResponseEntity**: پاسخ کامل HTTP.
- 20 **MultipartFile**: فایل آپلودی.
- 21 **WebDataBinder**: اتصال و اعتبارسنجی داده.
- 22 **ConversionService**: تبدیل نوع.
- 23 **Formatter / Converter**: قالب بندی و تبدیل.
- 24 **Validator / Errors / BindingResult**: اعتبارسنجی و خطاها.
- 25 **LocaleResolver / ThemeResolver**: مدیریت locale و theme.
- 26 **HandlerInterceptor**: میان گیر درخواست.
- 27 **WebMvcConfigurer**: پیکربندی MVC.

- 1 **SpringBootApplication**: ترکیب Configuration + ComponentScan + EnableAutoConfiguration.
- 2 **EnableAutoConfiguration**: فعال‌سازی پیکربندی خودکار.
- 3 **ComponentScan**: اسکن Component ها.
- 4 **Configuration**: کلاس پیکربندی.
- 5 **SpringApplication**: کلاس اجرای اپلیکیشن Boot.
- 6 **SpringApplicationBuilder**: سازنده اپلیکیشن Boot.
- 7 **ApplicationRunner / CommandLineRunner**: اجرای کد در شروع.
- 8 **ApplicationContextInitializer**: مقداردهی اولیه context.
- 9 **SpringApplicationRunListener**: شنونده چرخه اجرای Boot.
- 10 **Banner**: بنر شروع اپلیکیشن.
- 11 **spring.factories**: فایل ثبت auto-configuration.
- 12 **AutoConfiguration**: پیکربندی خودکار.
- 13 **Condition**: شرط فعال‌سازی Bean.
- 14 **Conditional** و مشتقاتش: شرط‌های مختلف برای فعال‌سازی.
- 15 **ImportSelector / DeferredImportSelector**: انتخاب **import** پویا.
- 16 **ImportBeanDefinitionRegistrar**: ثبت BeanDefinition سفارشی.
- 17 **SpringFactoriesLoader**: بارگذار فایل **spring.factories**.
- 18 **FailureAnalyzer / FailureAnalysis**: تحلیل خطاهای شروع.
- 19 **ExitCodeGenerator / ExitCodeExceptionMapper**: تولید کد خروج.
- 20 **SpringBootExceptionHandlerReporter**: گزارشگر خطاهای Boot.
- 21 **ApplicationContextFactory**: کارخانه ساخت context.
- 22 **ConfigurableApplicationContext**: context قابل پیکربندی.
- 23 **WebApplicationType**: نوع اپلیکیشن (Servlet/Reactive/None).

## ۲۳.۵ Spring Boot Actuator

- 1 **Actuator**: ماژول پایش و مدیریت اپلیکیشن.
- 2 **Endpoint**: نقطه پایانی مدیریتی.

- 3 @DeleteOperation / @WriteOperation / @ReadOperation / Endpoint: تعریف endpoint سفارشی.
- 4 Selector@: انتخابگر در endpoint.
- 5 @EndpointJmxExtension / EndpointWebExtension: افزونه‌های endpoint.
- 6 @JmxEndpoint / @ControllerEndpoint / @ServletEndpoint / RestControllerEndpoint: انواع endpoint.
- 7 EndpointAutoConfiguration@: پیکربندی خودکار endpoint ها.
- 8 HealthIndicator: نشانگر سلامت.
- 9 InfoContributor: تأمین‌کننده اطلاعات.
- 10 Metrics / MeterRegistry: متریک‌ها و ثبت‌کننده آن‌ها.
- 11 Counter / Gauge / Timer / DistributionSummary: انواع متریک.
- 12 AuditEventRepository: مخزن رویدادهای حسابرسی.
- 13 HttpTraceRepository: مخزن ردگیری HTTP.